
ECS 122A

Design and Analysis of Algorithms

v0.2

Jasper Lee

University of California, Davis

Contents

1	Introduction to Algorithm Analysis	1
1.1	Selection Sort by Iteration	1
1.1.1	Runtime	1
1.1.2	Correctness	1
1.1.3	The Purpose and Style of <i>Pseudocode</i>	3
1.2	Selection Sort by Recursion	3
1.2.1	Runtime	3
1.2.2	Correctness	4
1.3	What have we learnt?	4
2	Binary Search	6
2.1	Binary Search by Iteration	6
2.2	Searching in a Rotated Sorted Array	7
2.2.1	Challenge Problem: What about duplicates?	8
2.3	Single in Pairs	8
2.4	Binary Search: Connecting Optimization to Decision	9
2.4.1	Example: The Stick Cutting Problem	9
3	Divide and Conquer	11
3.1	Mergesort and Inversion Counting.	11
3.1.1	Counting Inversions	12
3.2	Longest All-Red Path in a Binary Tree	13
3.2.1	A First Approach	14
3.2.2	An Optimal Linear Time Algorithm	15
3.3	Closest Pair of Points in 2D.	16
3.4	Strassen's Algorithm for Matrix Multiplication	19
3.5	Integer Multiplication (Karatsuba's Algorithm)	20
3.6	Polynomial Multiplication	21
3.7	Convolution and Fast Fourier Transform.	21
3.7.1	Discrete Fourier Transform.	22
3.7.2	Fast Fourier Transform (FFT).	23

4	Dynamic Programming	25
4.1	Introduction to Dynamic Programming	25
4.1.1	Components of a (Standard) DP Algorithm	26
4.2	Counting Paths in a Grid	27
4.3	Maximum Subarray Sum	28
4.4	Answering Range Queries with Prefix Sums	30
4.5	Longest Common Subsequence.	30
4.6	0-1 Knapsack	31
4.7	Dynamic Programming on Trees.	32
4.7.1	Longest All-Red Path	32
4.7.2	Maximum Independent Set on Binary Trees	33
4.7.3	Maximum Matching in Binary Trees	34
4.8	Shortest Paths on Graphs.	35
4.8.1	Single-Source Shortest Path: Bellman-Ford.	35
4.8.2	Negative Cycles.	37
4.8.3	All-Pairs Shortest Path	38
5	Greedy Algorithms	39
5.1	Motivating Example: Fractional Knapsack	39
5.1.1	The General Proof Framework	40
5.1.2	Proving Correctness for Fractional Knapsack	41
5.2	Choose Your Team Tournament	42
5.2.1	Correctness	43
5.3	Data Compression and Huffman Coding	44
5.3.1	Problem Setting: Binary and Prefix-Free Codes	44
5.3.2	The Optimal Prefix-Free Binary Code Problem.	44
5.3.3	The Huffman Coding Algorithm	45
5.3.4	Correctness of Huffman Coding	46
5.3.5	Huffman Code Length and Discrete Entropy	47
5.4	Minimum Spanning Trees	48
5.4.1	Prim's Algorithm	48
5.4.2	Correctness	49
5.4.3	Implementation	50
5.5	Dijkstra's Algorithm	51
5.5.1	Shortest Path Tree	51
5.5.2	Algorithm.	52
5.5.3	Correctness	52

6	Reductions, NP-Hardness and Approximation Algorithms	54
6.1	Reductions	54
6.1.1	Example: Reduction from Optimization to Decision	55
6.1.2	Example: The Lineland Road Trip (Homework 3, Problem 3.2)	55
6.2	Computational Hardness	57
6.2.1	How to Show NP-Hardness	57
6.2.2	Known NP-Hard Problems	58
6.2.3	Hardness of Clique, Assuming Hardness of Independent Set	58
6.3	Complexity Classes	59
6.4	Approximation Algorithms	61
6.4.1	A 2-Approximation for 0-1 Knapsack	61
6.4.2	Fully Polynomial Time Approximation Schemes	62
6.5	Inapproximability	63
6.5.1	The Travelling Salesman Problem	63
6.5.2	Inapproximability of TSP	64
6.5.3	Metric TSP and a 2-Approximation	64

Introduction to Algorithm Analysis

We begin the course with an introduction to basic analysis techniques for algorithms. We consider selection sort as a running example, implemented by iteration and by recursion. We will see how to analyze both the correctness and runtime of these implementations.

1.1 Selection Sort by Iteration

Here is a straightforward iterative implementation of selection sort.

Algorithm 1.1. Selection Sort by Iteration

Input: Array A of length n , 1-indexed

1. for $i = 1$ to n :
 - (a) Find index $j \in [i..n]$ that minimizes $A[j]$
 - (b) Swap $A[i]$ and $A[j]$
-

1.1.1 Runtime

We will start with runtime analysis since that is much easier.

In the i^{th} loop iteration, the amount of work is $O(n - i)$. Summing over i from 1 to n gives $O(n(n - 1)/2) = O(n^2)$.

We could have even used a looser upper bound, that each loop iteration does $O(n)$ work instead of the tighter bound of $O(n - i)$.

1.1.2 Correctness

We will use induction to prove the correctness of [Algorithm 1.1](#). Induction is an extremely useful and important tool for analyzing algorithms and computation—it essentially keeps track of what is changing throughout the computation, allowing us to reason about the process.

To give an inductive proof, we need to *carefully* state the induction hypothesis.

INDUCTION HYPOTHESIS At the end of the i^{th} loop iteration, $A[1..i]$ contains the smallest i elements (of the original input) in sorted order **and** A is a permutation of the original input.

For loops/iterations, such induction hypothesis is sometimes also called a “loop invariant”—essentially, what remains true at each loop iteration (which is exactly what our induction hypothesis is also capturing).

At this point, you might be wondering, it seems that the first part of the induction hypothesis is the important part, why do we even need the second part (that A is a permutation of the original input)? As with many induction proofs, sometimes, we need to keep track of more information than what we are trying to prove, in order for the induction step to be provable. In the following proof, we will make sure to use *both* parts of the induction hypothesis assumed for the i^{th} iteration, and prove also *both* parts for the $i + 1^{\text{st}}$ iteration.

Let us now give the induction proof.

BASE CASE ($i = 0$, OR RIGHT BEFORE THE ITERATION BEGINS): The induction hypothesis is trivially true.

It is frequently the case that base cases of inductions are trivial and uninteresting. On the midterm and exam, if you do not write out boring base cases, you will not be penalized. The important thing is to spell out the interesting/core parts of the argument.

INDUCTION STEP: We will assume that the induction hypothesis is true for some $i \geq 0$, and now we need to show that the same holds for $i + 1$.

First, we will show that at the end of the $i + 1^{\text{st}}$ iteration, $A[1..(i + 1)]$ contains the smallest $i + 1$ elements in sorted order. To show this, let A_i be the version of the array at the end of the i^{th} iteration, or equivalently, the beginning of the $i + 1^{\text{st}}$ iteration. By the induction hypothesis, A_i is a permutation of the original input, while $A_i[1..i]$ contains the smallest i elements. Thus, the index j found in the $i + 1^{\text{st}}$ iteration must be for the $i + 1^{\text{st}}$ smallest element. The swap step ensures that $A[i + 1]$ does indeed contain the $i + 1^{\text{st}}$ smallest element at the end of the iteration. Furthermore, given that $A[1..i]$ is unchanged in the loop iteration, we can conclude that $A[1..(i + 1)]$ does indeed contain the smallest $i + 1$ elements in sorted order.

Next, we will show that A remains a permutation of the original input at the end of the $i + 1^{\text{st}}$ iteration. But this is trivial, since the only edit on the array is a swap.

The above two paragraphs complete the induction step.

WRAP UP: A lot of the times, after we complete an induction proof, we need to write a little bit more, to use the conclusion of the induction to show what we finally want. In this instance, it's pretty straightforward. The case $i = n$ from the induction hypothesis implies A is sorted at the end of the n^{th} iteration.

1.1.3 The Purpose and Style of Pseudocode

Before we move on to the recursive implementation of selection sort, let us briefly discuss the purpose of pseudocode and what it should look like.

Pseudocode is a means to communicate an algorithm to the reader—the emphasis is on *communication*.

In order to communicate effectively, a lot of the times it's more effective to use English to abstract out some algorithmic details, so that the high level structure of the algorithm becomes more apparent from the text. For example, [Algorithm 1.1](#) uses a single sentence describing the bigger step of finding the min in a sub-array, instead of writing out the minute details of implementing it. This makes the pseudocode much more readable and communicative.

It requires judgement and balance to decide the degree to which to use English over code. Generally, we want to consider how complex the overall problem and algorithm are. The more complex they are, the more leeway we have to abstract out lower level details (for example, [Algorithm 1.1](#) is a sorting algorithm, and we chose to abstract out min-finding). On the other hand, if the problem/algorithm is to find the min in an array, then of course we should not just write a single English sentence saying “we will find the min”.

The final thing to remember is that, when writing pseudocode, do not just invent a dialect of C or Python and write out a bunch of code. Think carefully what needs to be communicated.

1.2 Selection Sort by Recursion

Let us now consider a different implementation of selection sort, by recursion.

Algorithm 1.2. Selection Sort by Recursion

function SORT(A)

1. if $\text{size}(A)$ is 1, then return A , else
 2. Find $\arg \min_j A[j]$ ($\leftarrow$ This is mathematical notation to precisely state that we want to find the index j that minimizes $A[j]$)
 3. $B \leftarrow A$; $B[j] \leftarrow A[1]$;
 4. return $A[j] :: \text{SORT}(B[2 .. \text{size}(A)])$
-

1.2.1 Runtime

A common way to analyze the runtime of a recursive algorithm is to set up a *recurrence relation* (or recurrence *inequality*) and bound the growth of the algorithm runtime.

Let $T(n)$ be the (worst-case) runtime of SORT in [Algorithm 1.2](#), when given an array of size n . A call to SORT performs $O(n)$ work, and makes one recursive call to an array with one fewer element. Thus, $T(n)$ can be upper bounded by

$$T(n) \leq c_1 \cdot n + T(n - 1)$$

for some constant $c_1 \geq 1$, and without loss of generality we will assume that $T(1) = 1$ by saying that the $n = 1$ case takes “unit” time.

We want to show that SORT runs in $O(n^2)$ time, so it suffices to show that $T(n) \leq c_2 n^2$ for some sufficiently large constant c_2 . We will, you guessed it, again show this by induction.

BASE CASE: We know that $T(1) = 1$, so as long as $c_2 \geq 1$, the base case will hold.

INDUCTION STEP: Assuming $T(n-1) \leq c_2(n-1)^2$ for some $n > 1$, we have

$$\begin{aligned} T(n) &\leq c_1 n + T(n-1) \quad \text{by the recurrence inequality} \\ &\leq c_1 n + c_2(n-1)^2 \quad \text{by the induction hypothesis} \\ &= c_2 n^2 - 2c_2 n + c_1 n + c_2 \end{aligned}$$

If we choose $c_2 \geq c_1$ (and recalling that $n > 1$), then $c_1 n - 2c_2 n + c_2 \leq c_2 - c_2 n \leq 0$, implying that

$$T(n) \leq c_2 n^2$$

Thus, as long as $c_2 \geq \max(c_1, 1)$, then by induction we have shown that $T(n) \leq c_2 n^2$ for all $n \geq 1$, concluding that SORT runs in $O(n^2)$ time.

1.2.2 Correctness

As with the iterative version of selection sort, we will use induction to prove correctness. However, since the implementation is different, we will need to write a different proof, **with a different induction hypothesis**.

INDUCTION HYPOTHESIS: For every input array of size n , SORT returns the sorted version

We now proceed with the induction proof.

BASE CASE: Again, trivial for $n = 1$.

INDUCTION STEP: Assume the induction hypothesis for some $n \geq 1$, now consider giving an array of size $n + 1$ as input to SORT. We know that $A[j]$ is the smallest element in A , and $B[2..(n+1)]$ is some permutation of the n largest elements of A . Applying the induction hypothesis on $B[2..(n+1)]$, we conclude that $A[j] :: \text{SORT}(B[2..size(A)])$ is the sorted permutation of A .

WRAP UP: There really is not anything to wrap up. The induction hypothesis is exactly the statement we are trying to prove.

1.3 What have we learnt?

So what have we learnt from studying two different implementations of selection sort?

- The two implementation use two different induction variables. How do we choose generally?

- The key point is to choose an induction variable that *changes* during the computation. Induction as a proof technique keeps track of what is changing throughout the computation, and so the induction variable should also be changing.
- For loops and iterative algorithms, generally the induction variable is the loop index (even for while loops where there is no explicit loop index in the code or pseudocode).
- For recursive algorithms, typically the induction variable is input size, although there certainly are exceptions (that we will not go into, at least not in these notes).
- At a higher level, make sure that the proof structure follows and matches code structure. As a corollary, if we write spaghetti code with complicated structure, any analysis will necessarily be complicated and messy as well.
- Precision: notice that in the above examples, we never used any vague words like “correct” in the induction hypothesis or anywhere else in the proofs. We spelled out exactly what “correct” means in each context, to be precise about our claims.

Binary Search

In Chapter 1, we established basic algorithm analysis techniques. We now use it to analyze the classic binary search algorithm, searching over a sorted array. We will then see a few additional examples demonstrating that binary search is in fact a powerful tool, and a much more general paradigm than simply searching for values in a sorted array.

2.1 Binary Search by Iteration

SETTING: Given sorted array A , find the index of a given element x if x exists in the array.

Algorithm 2.1. Binary Search by Iteration

Assume $A[0..(n-1)]$ is a sorted array

1. $L \leftarrow 0, R \leftarrow n - 1$
 2. while $L \leq R$:
 - (a) $m \leftarrow L + \lfloor (R - L)/2 \rfloor$
 - (b) If
 - i. $A[m] < x$ then $L \leftarrow m + 1$
 - ii. $A[m] > x$ then $R \leftarrow m - 1$
 - iii. $A[m] = x$ then return m
 3. return “FAIL”
-

RUNTIME: $R - L$ is (roughly) halved each time, and so the algorithm runs in $O(\log n)$ time. Formally we can do an induction, but the formality required for these analyses depends on how convincing the informal argument is.

CORRECTNESS: This is an iterative implementation of binary search, so what should we induct on as the induction variable? Even though there is no explicit loop counter, we will still use the loop count.

The core of proving the correctness of a binary search implementation lies in properly defining the loop invariant: we guarantee that if the element we are looking

for exists, then it is always within the active search space $[L, R]$.

LOOP INVARIANT: At the end of loop iteration i , if $A[t] = x$ for some t , then $t \in [L, R]$.

The induction step is pretty straightforward, following the case analysis within binary search. Try it out yourself!

2.2 Searching in a Rotated Sorted Array

Consider the problem of finding the minimum element in a “rotated” sorted array.

SETTING: Let A be a sorted array of n **distinct** values. Suppose we are given A' , which is a cyclically shifted (rotated) version of A . Our goal is to find the index t corresponding to the original location of $A[0]$ (the minimum element of the array).

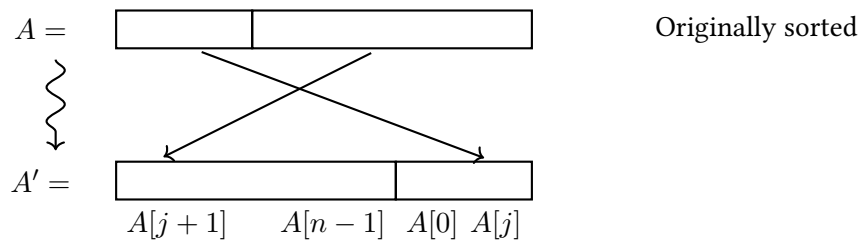


Figure 2.1. A rotated sorted array A' is formed by cyclically shifting blocks of the original sorted array A .

We can solve this problem using a modified binary search in $O(\log n)$ time.

NOTE ON IMPLEMENTATION DETAILS: For written homeworks and exams in this course, do not worry about off-by-one errors (e.g. whether a bound should be m or $m - 1$). You only need to worry about exact boundary arithmetic for coding assignments.

The core idea is to maintain a loop invariant and compare the midpoint to the endpoints to determine which half of the array contains t (location of the minimum array value, as seen in Figure 2.1).

LOOP INVARIANT: Throughout the execution of the loop, we always have $t \in [L, R]$. Consequently, the subarray $A'[L..R]$ is always a valid rotated sorted array containing the minimum element.

To update our bounds, we compare the midpoint element $A'[m]$ with the rightmost active element $A'[R]$ (we can also compare with $A'[L]$ instead for a symmetric but technically different algorithm). Because the elements are distinct, we have two cases:

1. **If $A'[m] > A'[R]$:** t must occur strictly after m , meaning the minimum element is in the right half. We update the bound to $L \leftarrow m + 1$.
2. **If $A'[m] < A'[R]$:** The right half from m to R is strictly increasing and so $[(m + 1)..R]$ does not contain t , so t is either m or to its left. We update the bound to $R \leftarrow m$.

2.2.1 Challenge Problem: What about duplicates?

A natural follow-up question is: What happens if we allow item duplicates in the rotated array?

It turns out the problem becomes significantly harder, requiring $\Omega(n)$ queries (and hence time) in the worst case. We will discuss the intuition here, but not give a fully formal proof.

Consider an adversary who constructs an array of length n consisting entirely of 1s, except for a single 0. The target minimum is the 0, and by rotation can be anywhere in the array.

It can be shown that any deterministic algorithm must make at least $n - 1$ queries in the worst case to guarantee finding the unique 0 in an array of 1s.

2.3 Single in Pairs

Let us now see another application of binary search outside of searching in a sorted array.

SETTING: We are given a sorted array A of n items, where n is an odd integer. Exactly one element appears uniquely in the array. All other $n - 1$ entries appear exactly twice as $(n - 1)/2$ adjacent pairs. Our goal is to find the lone unique element.

IDEA: For midpoint m , query $A[m - 1]$, $A[m]$, and $A[m + 1]$. Based on these queries, we can determine whether the unique element lies to the left or right of our current position by using a parity argument (as detailed in Figure 2.2).

3 possibilities

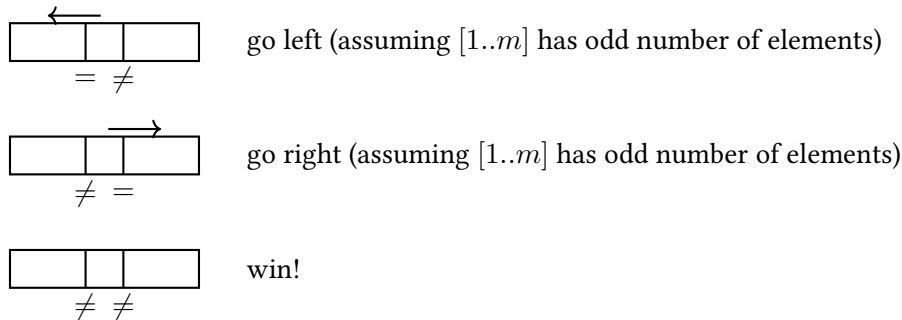


Figure 2.2. The three search possibilities based on local equality checks, assuming m is odd. If m is even, flip the directions.

RUNTIME: The runtime is clearly $O(\log n)$ since we halve the search space based on constant-time parity checks.

LOOP INVARIANT: $A[L..R]$ is an array containing the unique element, and all other elements within this subarray appear in pairs.

2.4 Binary Search: Connecting Optimization to Decision

The previous examples look basic and somewhat silly, but binary search is actually very powerful.

In particular, binary search connects (monotone) optimization problems to (monotone) decision problems, as we explain in this section.

Specifically, consider:

- **Problem 1 (Optimization):** Find the maximum integer k such that a property $P(k)$ is true. Here, we require $P(k)$ to be *monotonic*: if $P(k)$ is true for some k , it implies $P(k')$ for all smaller k' .
- **Problem 2 (Decision):** Given a fixed integer k' , output whether $P(k')$ is true or false.

These two problems turn out to be *equivalent* in a sense, that solving one implies we can solve the other.

The easy direction is Problem 1 $\rightarrow$ Problem 2: to solve Problem 2, just solve Problem 1 for the maximum k and compare with k' .

The more interesting direction we consider now: to solve Problem 1, use binary search over the function (or implicit array) $A[k] = \mathbb{1}\{P(k)\}$ over its possible domain.

2.4.1 Example: The Stick Cutting Problem

SETTING: We are given a list of n sticks with integer lengths $L_1, L_2, \dots, L_n$, and a target integer m . We can cut any stick of length ℓ into two sticks of integer lengths ℓ_a and ℓ_b such that $\ell_a + \ell_b = \ell$. These new sticks can be further divided. Our goal is to find the maximum integer length k such that we can produce at least m sticks of exactly length k , and it is ok if there are other leftover sticks of various lengths.

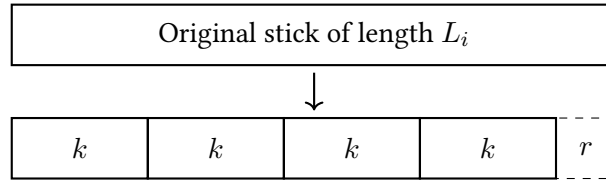


Figure 2.3. A stick of length L_i can be cut into pieces of length k , leaving some remainder r .

The above is an optimization problem, so what is the corresponding decision problem? It is:

$$P(k) = \{\text{We can produce } \geq m \text{ sticks of length exactly } k\}$$

We have to show that $P(k)$ is monotonic. Why does $P(k) \Rightarrow P(k')$ for $k \geq k'$? If $P(k)$ is true, we have successfully formed m sticks of length k . To form m sticks of length $k' < k$, we can simply trim off a piece of length $k - k'$ from each of the m sticks. Thus, the property holds for all smaller lengths.

Next, how do we check $P(k)$ for a fixed k ? For any original stick of length L_i , we can produce $\lfloor L_i/k \rfloor$ many sticks of length k . Therefore, to check $P(k)$, we compute:

$$\sum_{i=1}^n \left\lfloor \frac{L_i}{k} \right\rfloor$$

and return true if this sum is $\geq m$, and false otherwise. This check takes $O(n)$ time.

To solve the overall optimization problem, we binary search over the possible values of k . The domain of valid values of k is bounded between 1 and $L = \max(L_i)$.

NOTE: For homeworks and exams, once you establish a binary search over a monotonic array or property, there is no need to re-analyze the binary search itself.

RUNTIME ANALYSIS: The search space has size L , so the binary search requires $O(\log L)$ steps. In each step, we invoke our decision checker, which takes $O(n)$ time. The total runtime is therefore at most $O(n \log L)$.

CHAPTER 3

Divide and Conquer

This chapter explores the first major algorithmic paradigm in this course: Divide and Conquer.

There are predominantly two¹ different styles of divide and conquer algorithms:

1. Divide, then recurse into a single piece (e.g. binary search)
2. Divide, then work on every piece then combine (e.g. mergesort)

3.1 Mergesort and Inversion Counting

We will now look at examples of the second style, starting with mergesort. We recall the following recursive implementation of the algorithm.

Algorithm 3.1. Mergesort

function SORT(Array A of length n)

1. if $n \leq 1$, return A
 2. return MERGE(SORT($A[1 \dots \lfloor n/2 \rfloor]$), SORT($A[\lfloor n/2 \rfloor + 1 \dots n]$))
-

What mathematical claims and guarantees do we want from SORT and MERGE?

- SORT: on input a numerical array of size n , it should output a sorted permutation of the array in $O(n \log n)$ time.
- MERGE: on input two sorted arrays of sizes m and n , it should output a sorted version of their union/concatenation in $O(m + n)$ time.

We will not explicitly implement and analyze MERGE here, but instead assume the correctness and runtime of MERGE and use it to analyze SORT.

To analyze SORT, we will, of course, use induction. Here, the variable that is changing throughout the computation is the length “ n ” of the input array A , throughout the recursion.

The base case is straightforward: SORT clearly achieves the guarantees if A has length 0 or 1. For the induction step, observe that the output must be a permutation of A : the recursive calls of SORT output permutations of their respective halves by

¹Although in-between algorithms do exist.

the induction hypothesis, then MERGE preserves the existence of all elements by its correctness guarantee. Lastly, the output permutation must be sorted. This again follows from the induction hypothesis, that the recursive calls to SORT output sorted versions of the array halves, and the correctness of MERGE implies that combining these two sorted halves yields a fully sorted array.

3.1.1 Counting Inversions

Let us now go beyond mergesort and see a more advanced example: counting inverted pairs.

Definition 3.1 Given an array A , a pair of indices (i, j) is an *inversion* if $i < j$ and $A[i] > A[j]$. Note that i and j are not necessarily adjacent.

The trivial algorithm comparing all pairs runs in $O(n^2)$ time.

How do we apply the divide and conquer paradigm to design a faster algorithm? Here is the key observation.

Any pair $i < j$ *must* either be:

- Both in the first half of the array,
- Both in the second half of the array, or
- i being in the first half and j being in the second half.

This might sound silly and trivial, but this case analysis actually matches the structure of mergesort very well. The first and second cases can be handled by recursive calls like in mergesort, while the last case requires a variant of “merge”. The neat idea here, then, is to count the number of inverted pairs while executing mergesort itself.

Here is an adaptation of “merge” which, on input sorted arrays A and B , counts the number of inversions in $\text{CONCAT}(A, B)$.

Algorithm 3.2. Count Inversions During Merge

function COUNTINV(Array A of size m , Array B of size n) // Assuming A and B are sorted

1. $a_ind \leftarrow 1; b_ind \leftarrow 1; count \leftarrow 0;$
 2. while $a_ind \leq m$ and $b_ind \leq n$:
 - (a) if $A[a_ind] \leq B[b_ind]$
 - i. $a_ind \leftarrow a_ind + 1$
 - (b) else
 - i. $count \leftarrow count + (m - a_ind + 1)$
 - ii. $b_ind \leftarrow b_ind + 1$
 3. return $count$
-

We have a while loop in this function, so for proving correctness, we do induction over the loop variable (number of loop iterations). The induction variable and structure is implicit in the following induction hypothesis.

INDUCTION HYPOTHESIS: At the end of every loop iteration, the variable *count* holds the number of inversions in the (not computationally constructed) array $\text{CONCAT}(A, B[1..(b_ind - 1)])$. Moreover, *a_ind* and *b_ind* have been non-decreasing up till the end of this loop iteration.

INDUCTION STEP: Assume the induction hypothesis holds at the end of the previous loop iteration. We analyze both cases of the if-then-else.

- **Case 1:** *a_ind* increments. The claim for this induction step is just the previous induction hypothesis, so we are done.
- **Case 2:** *b_ind* increments, so we have something to actually prove. Let the value of *b_ind* before the loop iteration be *old_b_ind*, and the value after be *new_b_ind* = *old_b_ind* + 1. We know that $A[a_ind] > B[old_b_ind] = B[new_b_ind - 1]$, as this is the “else” case of the if-then-else. Since *A* is sorted, it follows that for all $k \in [a_ind, m]$, $A[k] \geq A[a_ind] > B[old_b_ind]$. If *a_ind* > 1, then it is also true that $A[a_ind - 1] \leq B[old_b_ind]$, because at the time *a_ind* - 1 was incremented to *a_ind*, we have $A[a_ind - 1] \leq B[prev_b_ind] \leq B[old_b_ind]$ where *prev_b_ind* was the *b* index at the time of *a_ind* increment. Therefore, if *a_ind* > 1, the number of new inversion pairs introduced from incrementing *b_ind* is exactly $m - a_ind + 1$. One can also easily check that the same count is correct for the edge case of *a_ind* = 1 as well.

WRAP UP: If the loop terminated with *b_ind* > *n*, then the induction hypothesis implies the claim about COUNTINV. But what if the loop terminates when *b_ind* ≤ *n*? This means the last increment must have been *a_ind*. So $A[m] \leq B[b_ind]$, and consequently everything in *A* is smaller than or equal to everything in $B[b_ind..n]$. So the number of inversions in $\text{CONCAT}(A, B)$ is equal to the number of inversions in $\text{CONCAT}(A, B[1..(b_ind - 1)])$, proving the claim.

3.2 Longest All-Red Path in a Binary Tree

Consider a binary tree of size *n*, where each vertex is colored either red or black. We want to find the longest length among all-red paths (where length is defined as the number of nodes).

To apply the Divide and Conquer paradigm, we observe a key structural property.

Observation 3.2 Any longest red path within a subtree rooted at some node *v* must either:

1. Be fully contained within the left child’s subtree,
2. Be fully contained within the right child’s subtree, or

Algorithm 3.3. Inefficient Longest All-Red Path

function DFS(Tree *node*)

1. if *node* is NULL or *node* is black, return 0
2. return $1 + \max(\text{DFS}(\text{node.left}), \text{DFS}(\text{node.right}))$

function LONGESTPATH(Tree *root*)

1. if *root* is NULL, return 0
 2. $\text{len_left} \leftarrow \text{LONGESTPATH}(\text{root.left})$
 3. $\text{len_right} \leftarrow \text{LONGESTPATH}(\text{root.right})$
 4. if *root* is red
 - (a) $\text{len_through_root} \leftarrow \text{DFS}(\text{root.left}) + 1 + \text{DFS}(\text{root.right})$
 5. else
 - (a) $\text{len_through_root} \leftarrow 0$
 6. return $\max(\text{len_left}, \text{len_right}, \text{len_through_root})$
-

3. Pass through the root *v* (which requires *v* to be red, and concatenates longest red paths starting downwards from the left and right children).

3.2.1 A First Approach

This structural observation naturally motivates a divide-and-conquer algorithm (Algorithm 3.3): recursively compute the longest red path lengths in the left and right subtrees for the first two cases, then handle the third case with DFS that traverses downwards from any given node to compute the longest red path length starting that node. Finally, we take the maximum across the three cases.

If we naively call this linear time DFS helper function on the current root at every recursive step, we obtain the following recurrence for the overall runtime $T(n)$:

$$T(n) \leq T(n_L) + T(n_R) + O(n)$$

where n_L and n_R are the sizes of the left and right subtrees, respectively, satisfying $n_L + n_R = n - 1$.

For *balanced* binary trees, where $n_L \approx n_R \approx n/2$, this yields the recurrence:

$$T(n) \leq 2T(\lceil n/2 \rceil) + O(n)$$

This is the same recurrence as for mergesort, which we know takes $O(n \log n)$ time.

For unbalanced trees, the runtime actually gets worse! For example, consider a “binary tree” that is just a single long linked list, meaning $n_L = n - 1$ and $n_R = 0$. The runtime recurrence becomes:

$$T(n) \leq T(n - 1) + O(n)$$

which solves to as much as $O(n^2)$ time, at least in badly behaved trees.

REMARK: Divide and Conquer algorithms (and algorithms generally) can take runtime that depends not only on the size of the input, but also on *how* the input is structured. This algorithm illustrates the dependence that can happen.

3.2.2 An Optimal Linear Time Algorithm

The bulk of the computation in the previous algorithm came from the 2 DFS calls within each node. To improve the algorithm down to optimal linear time, the observation is that the DFS calls have the same structure as the overall recursion, so the DFS calls are actually *redundant*.

For the new algorithm, instead of having the recursive call return one quantity (length of longest red path in subtree rooted at input vertex), we will ask it to compute *two* pieces of information:

1. Length of longest red path in the subtree rooted at the input vertex.
2. Length of longest red path in the subtree that has an endpoint at the input vertex.

By passing both values up the recursion, we can compute the longest red path length through the root (third case of case analysis) in $O(1)$ time rather than $O(n)$ time.

Algorithm 3.4. Linear Time Longest All-Red Path

function NEWLONGESTPATH(Tree *root*)

1. if *root* is NULL, return (0, 0)
 2. (*len_left*, *len_left_root*) $\leftarrow$ NEWLONGESTPATH(*root.left*)
 3. (*len_right*, *len_right_root*) $\leftarrow$ NEWLONGESTPATH(*root.right*)
 4. if *root* is red
 - (a) *len_through_root* $\leftarrow$ *len_left_root* + 1 + *len_right_root*
 - (b) *len_root* $\leftarrow$ max(*len_left_root*, *len_right_root*) + 1
 5. else
 - (a) *len_through_root* $\leftarrow$ 0
 - (b) *len_root* $\leftarrow$ 0
 6. return (max(*len_left*, *len_right*, *len_through_root*), *len_root*)
-

With this modification, the recursive work done at each node is strictly bounded by a constant, implying that the algorithm runs in $O(n)$ time.

While the above informal argument is sufficient for your homeworks and exams, more formally, we can always form the new runtime recurrence relation:

$$T(n) \leq T(n_L) + T(n_R) + O(1)$$

where again $n_L + n_R = n - 1$. By induction, it is straightforward to show that $T(n) \leq O(n)$.

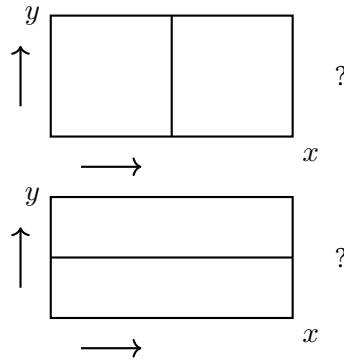
In fact, the above algorithm can be generalized for trees that are not only binary ones, which still runs in $O(n)$ time. There is a way to phrase the generalized algorithm such that, in each recursive call, the amount of work (in addition to making the recursive calls) is upper bounded by $O(\# \text{ children})$.

This also segues nicely to introducing a simpler way to argue about the runtime of recursive algorithms without resorting to the complete formality of induction. Notice that there is one recursive call per vertex in the tree, and the amount of work per vertex is $O(\# \text{ children})$. Thus, the total runtime is $O(\sum_v \# \text{ children for } v)$, but this sum is (up to constants) just the number of vertices in the tree! So we can conclude that the generalized algorithm has $O(n)$ runtime.

3.3 Closest Pair of Points in 2D

Given a list of n points (x_i, y_i) , we want to find the pair that is closest to each other, or at least, the distance of that closest pair. The trivial algorithm of trying all pairs is $O(n^2)$. We want to improve this to $O(n \log n)$ time.

For divide and conquer, we have to decide on how to divide. Since the problem lives on the x, y -plane, naturally we can choose to divide along either axis.



To yield an $O(n \log n)$ time algorithm, we really want to divide the n points in two halves of $n/2$ and $n/2$. We can achieve that by splitting points using the median x -coordinate (or y -coordinate, either works).

As we are seeing more and more examples of Divide and Conquer algorithms, hopefully a pattern is emerging. At the core of the algorithm, there is usually a key structure observation that enables the “conquer” step, and the observation is commonly a case analysis related to how we “divide”.

For the closest pair of points problem, we make the following observation. Assuming we split by the x -coordinate median, the closest pair must either be:

1. Both in the left half (solved by recursion).
2. Both in the right half (solved by recursion).
3. One point is in the left half, and one is in the right half (requires additional work for the “conquer” step).

At this point, we have an outline of what the algorithm should look like:

Algorithm 3.5. Closest Pair Algorithm Outline

function CLOSEST(List A)

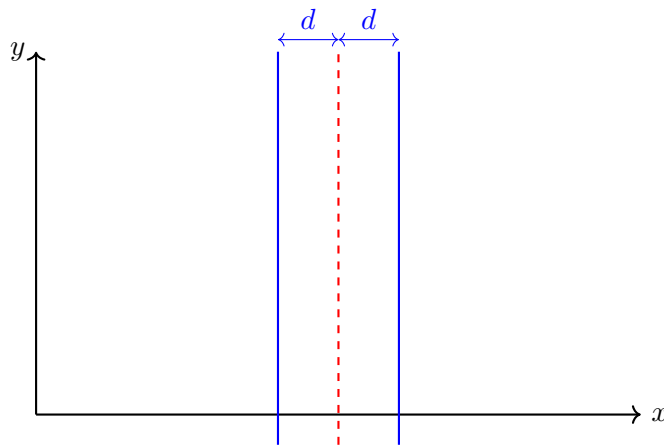
1. Find median in x , split into left and right halves.
 2. $d_L \leftarrow \text{CLOSEST}(\text{left half})$
 3. $d_R \leftarrow \text{CLOSEST}(\text{right half})$
 4. Compute d_{CROSS} for the cross-boundary case.
 5. return $\min(d_L, d_R, d_{\text{CROSS}})$
-

So then the big question is, how do we compute d_{CROSS} ?

If we compute d_{CROSS} naïvely, it takes $O(n^2)$ time *per recursive call* (although n does halve every call), making the total runtime at least $\Omega(n^2) \gg O(n \log n)$.

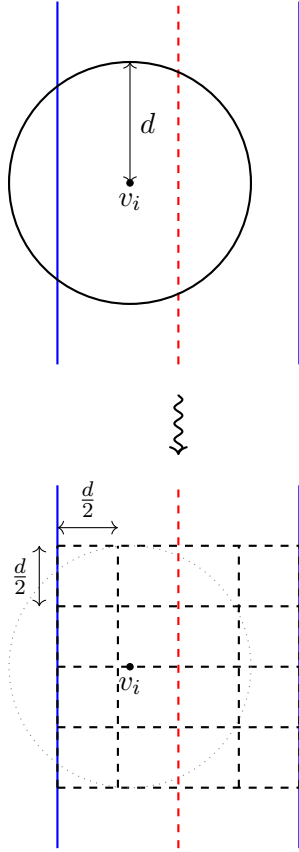
What we will see is a clever geometric algorithm that computes d_{CROSS} in $O(n)$ time, which yields an overall recursive $O(n \log n)$ time algorithm. The algorithm is highly non-trivial, but it is good to see an example of a harder algorithm.

Let $d = \min(d_L, d_R)$. Since we are only interested in pairs that are strictly closer than d , we only need to consider points within a vertical strip of x -median $\pm d$.



This basic observation is not enough yet, since every point might lie along this strip, so a naïve enumeration will still take $\Omega(n^2)$ time.

So here is the next observation. For any specific point v_i inside the strip, any point within distance d must lie in a radius- d ball around v_i . Meaning, for point v_i , we only care about the intersection of this ball with the strip.



We only need to consider the radius- d ball around v_i (specifically, its intersection with the strip).

This 4×4 grid covers the entire radius- d circle around v_i intersecting with the strip, so it is sufficient to check this area.

At most 1 point can lie in each square, since two points in the square have distance at most $d/\sqrt{2} < d$.

So we only need to consider the **15 closest points** in the y -coordinate within the strip.

In fact, observe that every pair (v_i, v_j) is being considered a duplicate second time as (v_j, v_i) . We can therefore optimize the 15-point check to a 7-point check, by sorting the points in the strip by their y -coordinates and pairing each point v_i with the next 7 points in the sorted list. This is almost an $O(n)$ merge step, except, the y -sorting requires $O(n \log n)$ time. This unfortunately yields an $O(n \log^2 n)$ time algorithm generally.

Algorithm 3.6. Intermediate $O(n \log^2 n)$ Closest Pair

function CLOSEST(List A)

1. Find median x -coordinate, split into left, right halves // Takes $O(n)$ time, but naïve sorting takes $O(n \log n)$ time
 2. $d_L \leftarrow \text{CLOSEST}(\text{left half})$
 3. $d_R \leftarrow \text{CLOSEST}(\text{right half})$
 4. Sort points in A by y -coordinate // Takes $O(n \log n)$ time
 5. New list $A' \leftarrow \text{filter } A \text{ to strip of } \min(d_L, d_R)$
 6. For each v_i in A' , try pairing with next 7 points to compute d_{CROSS}
 7. return $\min(d_L, d_R, d_{\text{CROSS}})$
-

If only we can sort in $O(n)$ time! (But that is impossible, at least for sorting algorithms based on comparing pairs of elements.)

The final observations are that: for the x -coordinate, we can just pre-sort the entire original input array; for the y -coordinate sorting, the recursive structure of CLOSEST is exactly the same as mergesort (noting that the “divide” step in mergesort can in fact arbitrarily bi-partition a list, instead of only considering the two contiguous halves). So the final algorithm would compute CLOSEST and mergesort in the y -coordinate all at the same time. It is an exercise for you to figure out the details.

3.4 Strassen’s Algorithm for Matrix Multiplication

From this section onwards, we move on to numerical and linear algebraic algorithms that are based on divide and conquer. One concrete difference is that, instead of the “case analysis” type observation we made for the previous combinatorial/geometric problems, here the structural observations will be algebraic.

First, we consider the matrix multiplication problem. Given two $n \times n$ matrices A and B , we want to compute their product $C = AB$. The naïve computation, which is how we do matrix multiplication by hand, takes $O(n^3)$ time.

Matrix multiplication inherently has a divide and conquer structure. The operation can be done in a blockwise manner as follows: interpreting A and B into a 2×2 matrix of matrix *blocks*, then the following holds.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}$$

This block formulation requires 8 matrix multiplications for size $\frac{n}{2} \times \frac{n}{2}$, so the runtime recurrence is $T(n) = 8T(\lceil n/2 \rceil) + c_1n^2$. Solving this recurrence unfortunately yields $T(n) = \Theta(n^3)$, which is not helping at all.

So why is the block structure useful? Strassen made an observation that is, frankly, magical. It turns out there is a way to compute 2×2 matrix multiplication using only 7 multiplications as opposed to 8!

First compute:

$$\begin{aligned} M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\ M_2 &= (A_{21} + A_{22})B_{11} \\ M_3 &= A_{11}(B_{12} - B_{22}) \\ M_4 &= A_{22}(B_{21} - B_{11}) \\ M_5 &= (A_{11} + A_{12})B_{22} \\ M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}) \\ M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}) \end{aligned}$$

Then the quadrants of C are reconstructed as:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{bmatrix}$$

Why does it matter that we only need 7 instead of 8 (matrix) multiplication recursive calls? The new recurrence is now $T(n) \leq 7T(\lceil n/2 \rceil) + cn^2$. It is not difficult to show by induction that $T(n) \leq c'n^{\log_2 7} - c''n^2$ (for some sufficiently large c' and c''). Noting that $\log_2 7 < \log_2 8 = 3$, Strassen's algorithm is strictly faster than $O(n^3)$ time.

Here is a final fun exercise. Given two complex numbers $a + bi$ and $c + di$, their product is $(ac - bd) + (ad + bc)i$. Naïvely, computing this product requires 4 real-number multiplications (ac , bd , ad , and bc). Can you compute this product using only 3 real-number multiplications?

3.5 Integer Multiplication (Karatsuba's Algorithm)

SETTING: Given two n -bit integers x and y , we want to compute their product $z = xy$. The standard algorithm for this is the one we learn in elementary school (applied to binary), which takes $O(n^2)$ time.

$$\begin{array}{r}
\begin{array}{cccc}
& & 1 & 1 & 0 & 1 \\
\times & & 0 & 1 & 0 & 1 \\
\hline
& & 1 & 1 & 0 & 1 \\
& 0 & 0 & 0 & 0 & \\
& 1 & 1 & 0 & 1 & \\
+ & 0 & 0 & 0 & 0 & \\
\hline
& 1 & 0 & 0 & 0 & 0 & 0 & 1
\end{array}
\end{array}$$

Can we do better, like Strassen's Algorithm?

The first step is to look at integer multiplication from a Divide and Conquer perspective. We can split the bits of x and y into halves:

$$x = x_{\text{low}} + 2^{\lfloor n/2 \rfloor} x_{\text{high}}$$

$$y = y_{\text{low}} + 2^{\lfloor n/2 \rfloor} y_{\text{high}}$$

where $x_{\text{low}} = x \bmod 2^{\lfloor n/2 \rfloor}$ (the bottom bits) and $x_{\text{high}} = x \operatorname{div} 2^{\lfloor n/2 \rfloor}$ (computed via a bit shift).

The product can be expanded as:

$$xy = x_{\text{low}}y_{\text{low}} + 2^{\lfloor n/2 \rfloor}(x_{\text{high}}y_{\text{low}} + y_{\text{high}}x_{\text{low}}) + 2^{2\lfloor n/2 \rfloor}x_{\text{high}}y_{\text{high}}$$

The simple method computes these 4 products $(x_{\text{low}}y_{\text{low}}, x_{\text{high}}y_{\text{low}}, y_{\text{high}}x_{\text{low}}, x_{\text{high}}y_{\text{high}})$ recursively, and then does bit shifts to compute the final product with $2^{\lfloor n/2 \rfloor}$ and $2^{\lceil n/2 \rceil}$. The recurrence for this simple method is:

$$T(n) \leq 4T(n/2) + O(n)$$

which yields $T(n) \leq O(n^2)$.

How do we improve this to 3 multiplications? This was the exercise from the last section, expressed in terms of complex numbers. We can use a simpler version of Strassen's trick:

- Compute $a = x_{\text{low}}y_{\text{low}}$
- Compute $b = x_{\text{high}}y_{\text{high}}$
- Compute $c = (x_{\text{low}} + x_{\text{high}})(y_{\text{low}} + y_{\text{high}})$

Then, the middle term can be recovered by

$$x_{\text{high}}y_{\text{low}} + y_{\text{high}}x_{\text{low}} = c - a - b$$

This reduces the algorithm to 3 recursive multiplications. The new recurrence is:

$$T(n) \leq 3T(n/2) + \alpha n$$

By induction, we can prove that $T(n) \leq \gamma n^{\log_2 3} - \beta n$ for some constant $\beta \gg \alpha$, which gives an overall runtime of $O(n^{\log_2 3})$.

3.6 Polynomial Multiplication

SETTING: Given two degree- n polynomials

$$p(x) = p_0 + p_1x + \cdots + p_nx^n$$

$$q(x) = q_0 + q_1x + \cdots + q_nx^n$$

Compute the product $(p \cdot q)(x)$.

We can use the exact same algorithm! Let $p_{\text{low}}(x)$ be the terms of p up to degree $\lfloor n/2 \rfloor$, and define the higher-degree terms as:

$$p_{\text{high}}(x) = \frac{p(x) - p_{\text{low}}(x)}{x^{\lfloor n/2 \rfloor + 1}}$$

We can express the product $p \cdot q$ identically to integer multiplication:

$$p \cdot q = p_{\text{low}}q_{\text{low}} + x^{\lfloor n/2 \rfloor + 1}(p_{\text{low}}q_{\text{high}} + q_{\text{low}}p_{\text{high}}) + x^{2\lfloor n/2 \rfloor + 2}p_{\text{high}}q_{\text{high}}$$

Using the same 3-multiplication trick, this has exactly the same structure as Karatsuba's algorithm before, so we have an $O(n^{\log_2 3})$ algorithm.

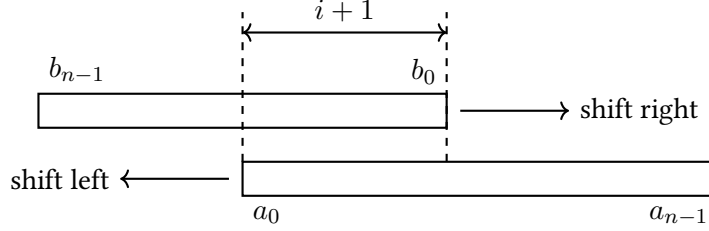
Can we do even better? **YES!** We can improve this basically to $O(n \log n)$.

3.7 Convolution and Fast Fourier Transform

SETTING: Given two length- n sequences a and b , compute the convolution $a * b$ (of length $2n - 1$).

$$(a * b)_i = \sum_{j=0}^i a_j b_{i-j}$$

Visually, this can be understood as reversing array b and “swiping” it across array a . $(a * b)_i$ corresponds to an overlap of width $i + 1$ and computing the dot product across the overlapping elements.



The naïve way to compute this takes $O(n^2)$ time: $O(n)$ terms each taking $O(n)$ work (for the dot product). We can improve this to $O(n \log n)$ time using the *Fast Fourier Transform* (FFT) algorithm.

3.7.1 Discrete Fourier Transform

Before discussing the Fast Fourier Transform which is an algorithm, we need to introduce the Discrete Fourier Transform (DFT) which is the mathematical operation that the FFT computes.

Given a length- n sequence (vector) v , the Discrete Fourier Transform of v is the matrix-vector product $\mathcal{F}_n v$, where $\mathcal{F}_n$ is defined as the following $n \times n$ matrix:

$$\mathcal{F}_n = \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \dots & \omega^{(n-1)^2} \end{pmatrix}$$

where ω is a primitive n -th root of unity (i.e. $\omega^n = 1$ and ω^i generates all n -th roots of unity), for example $\omega = e^{-2\pi i/n}$. Computing the DFT naïvely via standard matrix-vector multiplication $\mathcal{F}_n v$ takes $O(n^2)$ time.

Taking Fourier Transforms are standard procedures, with different standard libraries containing implementations.

How are Discrete Fourier Transforms related to convolutions? To discuss this, we need to first introduce the inverse transform $\mathcal{F}_n^{-1}$, which is mathematically very similar. It is essentially the same as the forward Fourier Transform, but instead using the inverse of ω and scaling by $\frac{1}{n}$:

$$\mathcal{F}_n^{-1} = \frac{1}{n} \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \dots & \omega^{-(n-1)} \\ 1 & \omega^{-2} & \omega^{-4} & \dots & \omega^{-2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-(n-1)} & \omega^{-2(n-1)} & \dots & \omega^{-(n-1)^2} \end{pmatrix}$$

The Discrete Fourier Transform has a powerful and convenient property, relating to convolutions: convolution in the normal domain corresponds to pointwise multiplication in the Fourier domain²:

$$a * b = \mathcal{F}_n^{-1} ((\mathcal{F}_n a) \cdot (\mathcal{F}_n b))$$

Since pointwise multiplication of two sequences takes linear time, if we can compute forward and inverse DFTs efficiently, we can compute convolutions efficiently. The Fast Fourier Transform (FFT) algorithm, described in the next section, lets us compute DFTs and hence convolutions in $O(n \log n)$ time. But an important note is that the Fourier Transform can have large numerical (relative) errors; in fact, MATLAB implements convolutions naïvely (last time I checked) instead of using the FFT.

So why is this useful for polynomial multiplication? It turns out polynomial multiplication is just a convolution:

$$\begin{aligned} p \cdot q &= p * q \\ (p \cdot q)(x) &= \sum_i x^i \sum_j p_j q_{i-j} \end{aligned}$$

That means we can also do polynomial multiplication in $O(n \log n)$ time, using FFT below.

3.7.2 Fast Fourier Transform (FFT)

The Fast Fourier Transform (FFT) is a standard divide and conquer algorithm that leverages the structure of the DFT matrix and computes $\mathcal{F}_n v$ in $O(n \log n)$ time, as opposed to the slower naïve $O(n^2)$ time.

By definition, the k^{th} element of the DFT is:

$$(\mathcal{F}_n v)_k = \sum_{j=0}^{n-1} v_j e^{-i \frac{2\pi k j}{n}}$$

We can split the sum into even and odd indices, and then manipulate the two sums slightly:

$$\begin{aligned} (\mathcal{F}_n v)_k &= \sum_{j=0}^{\frac{n}{2}-1} v_{2j} e^{-i \frac{2\pi k (2j)}{n}} + \sum_{j=0}^{\frac{n}{2}-1} v_{2j+1} e^{-i \frac{2\pi k (2j+1)}{n}} \\ &= \sum_{j=0}^{\frac{n}{2}-1} v_{2j} e^{-i \frac{2\pi k j}{n/2}} + e^{-i \frac{2\pi k}{n}} \sum_{j=0}^{\frac{n}{2}-1} v_{2j+1} e^{-i \frac{2\pi k j}{n/2}} \end{aligned}$$

²Strictly speaking, computing this with an $n \times n$ DFT matrix yields the *circular* convolution different from the *linear* convolution above, so the equivalence is technically false. But this can be made true via sufficient zero-padding, so the claim as stated is “morally true” and is representative of the general duality between convolution and pointwise multiplication in the normal and Fourier domains.

Now the key observation is that the two sums are exactly the k^{th} elements of the DFTs of the even and odd halves of v (up to mapping $k \mapsto k \bmod (n/2)$), each of length $n/2$, which can be handled by recursive FFT calls. In other words, computing the DFT of size n requires exactly two DFTs of size $n/2$, and a final pointwise multiplication by $e^{-i\frac{2\pi k}{n}}$ (taking $O(n)$ time).

The runtime recurrence relation is

$$T(n) \leq 2T(n/2) + O(n)$$

which implies $T(n) \leq O(n \log n)$.

Dynamic Programming

This chapter introduces the powerful algorithmic design paradigm of dynamic programming, which stems from the simple-sounding observation that repeated computation can be optimized away by storing the results.

4.1 Introduction to Dynamic Programming

Let us use recursion as a starting point. Generically, recursion breaks down a larger problem into subproblems of the same kind, and recurses until the computation reaches base cases.

In prior Divide and Conquer algorithms we saw, the subproblems never overlap with each other. Every recursive call is handling a different subproblem or a different part of the input. On the other hand, the core idea of Dynamic Programming is to structure the recursion in a way such that the subproblems can overlap, and the key to the computation is to store the subproblem results so they can be reused efficiently without re-computation.

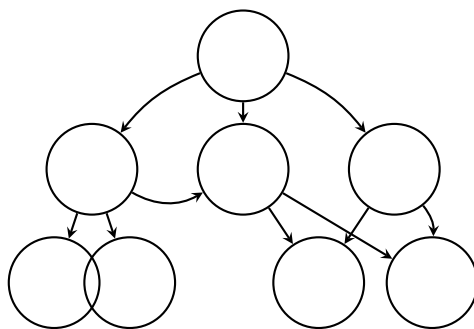


Figure 4.1. A recursion tree with overlapping subproblems. This is formally a directed acyclic graph.

As a simple but informative example, consider the Fibonacci sequence, defined by the recurrence relation $F(n+2) = F(n+1) + F(n)$, with base cases $F(0) = 0$ and $F(1) = 1$. If we implement this definition directly as a recursive function, the runtime grows proportionally to the Fibonacci numbers themselves, resulting in an exponential $\Theta(F(n))$ time complexity.

Instead of running an exponential time recursion, we can in fact just store the Fibonacci numbers in an array, and compute them using a simple for loop. This is a very simple instance of a dynamic programming algorithm.

Algorithm 4.1. Bottom-Up Fibonacci

1. Initialize array $F \leftarrow$ zeros of size $n + 1$
 2. $F[0] \leftarrow 0$; $F[1] \leftarrow 1$
 3. for $i = 2$ to n :
 - (a) $F[i] \leftarrow F[i - 1] + F[i - 2]$
 4. return $F[n]$
-

This algorithm runs in $O(n)$ time and $O(n)$ space. In fact, we can further optimize the space complexity to $O(1)$ by observing that we only ever need the two most recently computed values.

4.1.1 Components of a (Standard) DP Algorithm

When designing a standard Dynamic Programming algorithm, there are four core components we must precisely specify:

1. **Table entries:** Dynamic programming stores computation results, which we can generically consider as a table, where each entry can be considered a subproblem of the original problem (after some transformation). To specify the table, we have to state the precise meaning of the table entries and how the meaning depends on the indices of the entries.
2. **Recurrence:** How are the table entries mathematically related to one another?
3. **(Pseudo)code to fill the table:** There are typically two ways (*bottom-up* vs *top-down*) to actually implement a dynamic programming algorithm, once the table and the recurrence are specified. The Fibonacci implementation above is bottom-up via iteration, and right below we will see a top-down implementation via recursion.
4. **Wrap up:** How can the final answer to the original problem be computed from the fully-filled table?

Here we give a top-down implementation for the Fibonacci numbers. Essentially, we just implement the recurrence relation directly via recursion, and store the subproblem results in a global table. At the beginning of each recursive call, we check whether the subproblem has already been computed; if yes, simply return the table entry, otherwise we proceed with the recursive computation.

Algorithm 4.2. Top-Down Fibonacci

Global variable: array $F[0 \dots n]$ initialized to -1

function FIB(n)

1. if $F[n] \neq -1$: return $F[n]$
2. if $n = 0$: $F[0] \leftarrow 0$; return 0
3. if $n = 1$: $F[1] \leftarrow 1$; return 1
4. $F[n] \leftarrow \text{FIB}(n - 1) + \text{FIB}(n - 2)$
5. return $F[n]$

4.2 Counting Paths in a Grid

Consider an $m \times n$ grid where some cells are blocked, and we wish to count the total number of valid paths from the start cell $(1, 1)$ to the end cell (m, n) . The allowable moves are strictly restricted to moving right or moving down, namely, moving from (i, j) to $(i + 1, j)$ or $(i, j + 1)$.

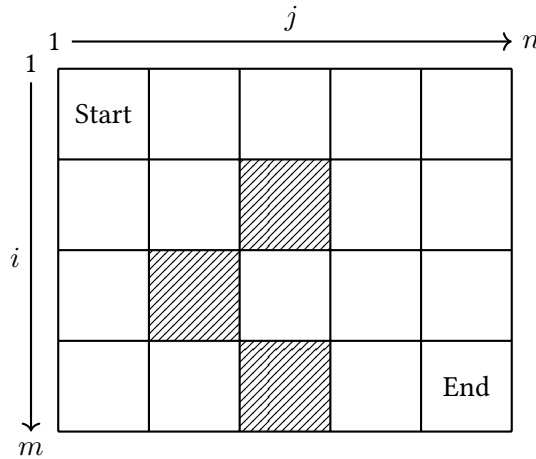


Figure 4.2. An $m \times n$ grid containing explicitly blocked cells. The goal is to traverse from the top-left to the bottom-right using only valid down and right moves.

To apply Dynamic Programming, we specify the four components.

TABLE ENTRIES: What are possible table structures and precise table entry meaning? Perhaps the obvious thing to try is to follow the grid structure, with a table indexed by (i, j) . In terms of the *meaning* of the table entry, we can try directly generalizing the overall problem: Let $T(i, j)$ be the number of valid paths from $(1, 1)$ to (i, j) .

RECURRENCE: How do these entries relate? Any valid path reaching (i, j) **must** make its final step from either the cell directly above, $(i - 1, j)$, or the cell directly to the

left, $(i, j - 1)$ ¹. The numbers of such paths are $T(i - 1, j)$ and $T(i, j - 1)$ respectively. These two sets of paths are also mutually exclusive, so we can conclude:

$$T(i, j) = T(i - 1, j) + T(i, j - 1)$$

for non-blocked cells. Also, $T(1, 1) = 1$, and for any cell that is blocked or have out-of-bounds indices (where $i = 0$ or $j = 0$), $T(i, j) = 0$.

PSEUDOCODE: Here is an implementation.

Algorithm 4.3. Count Grid Paths

1. Initialize table $T[0..m, 0..n]$ with zeros.
 2. $T[1, 1] \leftarrow 1$
 3. for $i = 1$ to m :
 - (a) for $j = 1$ to n :
 - i. if $(i, j) = (1, 1)$, continue
 - ii. if (i, j) is blocked, $T[i, j] \leftarrow 0$
 - iii. else $T[i, j] \leftarrow T[i - 1, j] + T[i, j - 1]$
 4. return $T[m, n]$
-

RUNTIME: The nested loops clearly take $O(mn)$ time. There are $O(mn)$ table entries, and each entry takes $O(1)$ computation time to compute the recurrence.

CORRECTNESS: We already showed that the DP table recurrence is true. The correctness of the computation follows from an implicit induction argument: the order of our double for-loop ensures that by the time we compute $T[i, j]$, the subproblems $T[i - 1, j]$ and $T[i, j - 1]$ have already been computed. This type of sentence is all you need to say for homeworks and exams.

4.3 Maximum Subarray Sum

Suppose we are given an array $A[1..n]$, and we wish to compute the maximum sum of any contiguous subarray, denoted mathematically as $\max_{i,j} \sum(A[i..j])$. Note that the chosen i can be strictly greater than j , denoting an empty sum. This can happen when the entire array consists of negative numbers.

A naïve algorithm tries all possible pairs of subarray boundaries (i, j) and computes the sum for each subarray, yielding $O(n^3)$ runtime². Our goal is to improve this to $O(n)$ time using dynamic programming.

¹This is like the kind of case analysis we did for Divide and Conquer, making a precise claim and argument about covering all cases of what solutions look like.

²Although improvable to $O(n^2)$ time using the prefix sum technique in Section 4.4.

Sometimes, the target runtime of the dynamic programming algorithm can give hints as to what the table can look like. Since we restrict ourselves to a $O(n)$ time algorithm, the DP table can only be $O(n)$ in size, as each entry must take at least some unit time to process.

Given that, for our first try, let us consider a table structure $T_{\max}[i]$ for $i \in [1..n]$. What is a reasonable table entry definition for this single row DP table? We can, again, try directly generalizing the overall problem, by *restricting the input* considered. Specifically, let us define

$$T_{\max}[i] = \text{the maximum subarray sum of the prefix } A[1..i].$$

Given this DP table definition, we can now try deriving a corresponding recurrence. To do so, we again need to make an observation on the structure of an optimal solution, covering all possible cases. Here, any optimal solution to $T_{\max}[i]$ **must** either be empty, or it does not use $A[i]$ in the sum, or it does use $A[i]$ in the sum. In the non-empty case where $A[i]$ is not used, the optimal solution must also be optimal for the subproblem of $T_{\max}[i - 1]$. However, if it does use $A[i]$, then we are left with many possibilities still. All we know is that the desired solution does use $A[i]$, so we have to consider $A[i]$, $A[(i - 1)..i]$, $A[(i - 2)..i]$ all the way to $A[1..i]$. This gives a long recurrence

$$\begin{aligned} T_{\max}[i] = \max(0, & T_{\max}[i - 1], \text{sum}(A[i]), \\ & \text{sum}(A[(i - 1)..i]), \\ & \text{sum}(A[(i - 2)..i]), \\ & \vdots \\ & \text{sum}(A[1..i])) \end{aligned}$$

This is indeed a dynamic programming algorithm, but if we had implemented this, the runtime would be $\Omega(n^2)$ since table entry $T_{\max}[i]$ requires at least $\Omega(i)$ computation, even if the sums can each be evaluated instantly in unit time (which they can be if the algorithm is appropriately structured).

Our next observation is that, the bulk of the work inside the $T_{\max}[i]$ recurrence is actually solving a slightly different algorithmic problem, which itself can be tackled by dynamic programming! Specifically, we needed to compute the maximum subarray sum of $A[1..i]$ that *has* to use $A[i]$. Define this to be $T_{\text{suffix}}[i]$. The structural observation for the recurrence of $T_{\text{suffix}}[i]$ is similar to the one for $T_{\max}[i]$, that any such optimal solution must either use $A[i - 1]$ or not. In the “not” case, the subarray is only $A[i]$. In the “yes” case, any optimal solution for $T_{\text{suffix}}[i]$ that uses $A[i - 1]$ might as well use an optimal solution for $T_{\text{suffix}}[i - 1]$, yielding a value of $T_{\text{suffix}}[i - 1] + A[i]$. Summarizing, T_{suffix} has the recurrence

$$T_{\text{suffix}}[i] = \max(A[i], T_{\text{suffix}}[i - 1] + A[i])$$

Now it is straightforward to compute T_{suffix} in $O(n)$ time. With T_{suffix} computed, we can write down a new recurrence for the original $T_{\max}$ table as

$$T_{\max}[i] = \max(0, T_{\max}[i - 1], T_{\text{suffix}}[i])$$

which of course now takes only $O(n)$ time to compute as well.

4.4 Answering Range Queries with Prefix Sums

Here is a simple but incredibly useful trick enabled by dynamic programming.

SETTING: Given an array $A[1 \dots n]$, we wish to prepare a data structure to answer range queries $(i, j) \mapsto \sum_{k=i}^j A[k]$.

A naïve algorithm just evaluates each query directly, using $O(j - i)$ time per query.

We can improve this significantly by observing that contiguous sums can be expressed as the difference between *prefix sums*:

$$\sum_{k=i}^j A[k] = \sum_{k=1}^j A[k] - \sum_{k=1}^{i-1} A[k]$$

We can precompute the prefix sums of A in $O(n)$ time.

TABLE ENTRIES: $T[i] = \sum_{k=1}^i A[k]$

RECURRENCE: $T[i + 1] = T[i] + A[i + 1]$

After this $O(n)$ preprocessing step, any arbitrary range query (i, j) can be answered in $O(1)$ time simply by evaluating $T[j] - T[i - 1]$.

4.5 Longest Common Subsequence

SETTING: Given a sequence S , a subsequence is formed by deleting some symbols from a sequence without changing the relative order of the remaining symbols. For example, if $S = \text{"abcdefg"}$, then “adf”, “adfg”, “bce” are all valid subsequences. Generally, for a sequence of length n , there are always 2^n many subsequences (including the empty one, and including double-counting sequences when there are repeated symbols).

Given two sequences $A[1 \dots m]$ and $B[1 \dots n]$, our goal is to find the length of the longest common subsequence (LCS) appearing in both A and B .

Naïve exhaustive search would take time proportional to $\exp(m) \exp(n)$. Instead, we will derive an algorithm that runs in $O(mn)$ time. As an aside, a recently-developed research subfield of fine-grained complexity conjectures that there is no $O(m^{.999} n^{.999})$ time algorithm. The LCS is known to be a “SETH-hard” problem, where SETH stands for Strong Exponential Time Hypothesis: a hypothesis that is stronger than $P \neq NP$ but which is also believed by many researchers.

Given our target runtime, we can afford a table of size $O(mn)$. We will try the approach from previous examples, which is to define table entries as the LCS problem itself, but on a restricted subset of the input.

TABLE ENTRIES: Let $T[i, j]$ be the length of the LCS for the prefixes $A[1..i]$ and $B[1..j]$.

RECURRENCE: Any longest common subsequence must either include the characters $A[i]$ and $B[j]$ (if they are equal symbols), or it does not use $A[i]$, or it does not use $B[j]$. Taking the maximum among these exhaustive cases yields the recurrence:

$$T[i, j] = \max \begin{cases} 1 + T[i - 1, j - 1] & \text{if } A[i] = B[j] \\ T[i - 1, j] \\ T[i, j - 1] \end{cases}$$

(Note: If $A[i] = B[j]$, it is always optimal to match them, but expressing the recurrence as a maximum over all branches is safe and easier to prove correct, as we did above.)

PSEUDOCODE: The DP table can be filled using a double loop, just like in the Section 4.2 example.

WRAP UP: The final answer is found at $T[m, n]$.

Given this example on longest common subsequence, you should try working out the similar problem of longest common substring.

EXERCISE: Design an $O(mn)$ time algorithm to find the longest common *substring*, where a substring is a *contiguous* subsequence.

4.6 0-1 Knapsack

SETTING: We have a knapsack with an integer capacity C . We are given a list of n items, where each item i has a reward r_i and an integer weight w_i . We want to find a subset of items such that their total weight is at most C and their total reward is maximized. The “0-1” in the problem name means that we can either take an item or not; there is no way to take an item fractionally, nor is there a way to take multiple copies of an item.

This is one of many different formulations of the knapsack problems in the wild. For example, other formulations can assume integer rewards instead of integer item weights/knapsack capacity. Different formulations will require different algorithms, even if they share some conceptual commonalities.

We have not really discussed greedy algorithms yet, but would being greedy work for this problem? No!

For example, a natural greedy algorithm would be to choose items in descending order of r_i/w_i , namely the “reward density”. But here is a simple bad example: capacity $C = 10$, item 1 has $r_1 = 2, w_1 = 1$, item 2 has $r_2 = 10, w_2 = 10$.

We will instead solve the problem using dynamic programming.

Our first attempt is to replicate the strategy before, defining a DP table based on the same problem but on restricted inputs:

$$T_0[i] = \text{best 0-1 knapsack solution on items } 1..i$$

But unfortunately this essentially cannot work, because the capacity constraint is not included in the problem definition, and it is insufficient for all the subproblems to have the same capacity C .

So in our next attempt, we instead try a 2-dimensional DP table.

TABLE ENTRIES: Let $T[i, c]$ be the maximum reward for 0-1 knapsack problem on items $1..i$ under a maximum capacity c . Here, the “remaining” capacity of the knapsack is information that requires tracking, and we use a table index to do so. This is a general technique that is frequently (but not always) useful.

RECURRENCE: An optimal subset of items either uses item i , or it does not. If it does not, then the subproblem is simply restricted to items $1..(i - 1)$ with the same capacity c , corresponding to table entry $T[i - 1, c]$. If it does use item i , then after excluding item i , this reduces to a 0-1 knapsack instance on items $1..(i - 1)$ but now with capacity $c - w_i$. This reasoning yields the recurrence:

$$T[i, c] = \max(r_i + T[i - 1, c - w_i], T[i - 1, c])$$

Here, $T[i, c]$ is defined to be $-\infty$ if $c < 0$.

COMPUTATION: A double for-loop, for example, with i as outer loop and c as inner loop (or vice versa).

WRAP UP: The solution to our original problem, by definition of our subproblem formulation, is at $T[n, C]$.

This algorithm takes $O(Cn)$ time, and also $O(Cn)$ space to store the entire 2-dimensional DP table. But we can actually do better on the space front. Observe that, in order to compute the row $T[i, \cdot]$, we only need the row $T[i - 1, \cdot]$. By keeping only one row, we can optimize the memory usage to $O(C)$ space only.

4.7 Dynamic Programming on Trees

Dynamic programming can apply beyond the rectangular table structure in the previous examples. Another common DP table structure is to follow an input tree.

We have actually already seen instances of this when studying Divide and Conquer algorithms, in the Longest All-Red Path example recapped below. Generally, dynamic programming on trees does frequently incorporate elements of Divide and Conquer, given the natural recursive structure of trees.

4.7.1 Longest All-Red Path

We revisit the problem of finding the longest all-red path in a binary tree, formalizing it through the lens of dynamic programming.

TABLE ENTRIES: We define two state variables for each node:

- $Len[node]$: the length of the longest all-red path strictly within the subtree rooted at $node$.

- $Len_root[node]$: the length of the longest all-red path that starts at $node$ and lies entirely within the subtree rooted at $node$.

RECURRENCE:

$$Len_root[node] = \begin{cases} 1 + \max(Len_root[node.left], Len_root[node.right]) & \text{if } node \text{ is red} \\ 0 & \text{otherwise} \end{cases}$$

$$Len[node] = \max \left(\begin{array}{l} Len[node.left], Len[node.right], \\ \begin{cases} 1 + Len_root[node.left] + Len_root[node.right] & \text{if } node \text{ is red} \\ 0 & \text{otherwise} \end{cases} \end{array} \right)$$

The recurrence can even be generalized to trees that are not binary, which is left as an exercise. For each DP table entry (namely, for each node), the recurrence should be computable in $O(\# \text{ children})$ time. Then, summing over all nodes, the total runtime will be $O(\text{tree size})$ which is linear time.

4.7.2 Maximum Independent Set on Binary Trees

SETTING: Given a binary tree T , the goal is to choose the maximum number of vertices such that no edge has both of its endpoints chosen.

What is the core “either/or” or “cover all cases” observation here? For this problem, it is again a simple observation: for any vertex in the tree, an independent set will either include that vertex, or it will not.

We will again use a DP table that follows the input tree structure instead of a rectangular table.

TABLE ENTRIES: Let us try the obvious table definition:

$A[node]$ = the size of the maximum independent set in the subtree rooted at $node$

How do we write a recurrence for this DP table? If a solution does not take $node$, then it might as well take best solutions for its children, the union of which is a valid independent set. If it does take $node$, it cannot take any of $node$'s children, but it could take the grandchildren, so it might as well take the best independent sets for each grandchildren subtree, whose union with $\{node\}$, again, is indeed a valid independent set.

RECURRENCE:

$$A[node] = \max \left\{ \begin{array}{l} A[node.left] + A[node.right] \quad (\text{not taking } node) \\ \begin{array}{l} 1 + A[node.left.left] \\ + A[node.left.right] \\ + A[node.right.left] \\ + A[node.right.right] \end{array} \quad (\text{taking } node) \end{array} \right.$$

RUNTIME: The work per node is $O(\# \text{ children} + \# \text{ grandchildren})$. Summing over the tree yields $O(\text{tree size})$ total work.

While the first attempt works, there is in fact an alternative cleaner dynamic programming approach that avoids looking at grandchildren, using a slightly augmented table structure.

TABLE ENTRIES: Let $A[\text{node}, \text{can_take}]$ where can_take is a Boolean, defined as

- If $\text{can_take} = 1$, it stores the max independent set size for the subtree rooted at node .
- If $\text{can_take} = 0$, it stores the max independent set size for the subtree rooted at node with the restriction that node **cannot** be included.

RECURRENCE: The above DP table definition yields a much cleaner recurrence.

$$\begin{aligned} A[\text{node}, 0] &= A[\text{node.left}, 1] + A[\text{node.right}, 1] \\ A[\text{node}, 1] &= \max(A[\text{node}, 0], 1 + A[\text{node.left}, 0] + A[\text{node.right}, 0]) \end{aligned}$$

The correctness analysis is largely the same as the previous algorithm, but this formulation requires only looking at immediate children.

EXERCISE: Can you generalize both dynamic programming approaches to trees that are not binary? Again, the goal is that the recurrence should only induce work proportional to the number of children (or grandchildren), so that summing over the tree, the total amount of work is linear in the tree size.

4.7.3 Maximum Matching in Binary Trees

Definition 4.1 A matching (in a graph) is a subset of edges such that each vertex is incident to at most one edge in the matching.

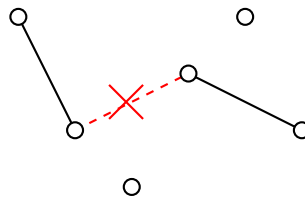


Figure 4.3. Visualizing the vertex constraint of a matching. Solid black edges form a valid matching because they share no vertices. The dashed red edge cannot be further included because it would result in a vertex being incident to more than one edge.

In this section, our algorithm task is to compute the maximum size across all matchings, in $O(n)$ time for a binary tree.

The key structural “either or” observation is that for any fixed edge, a matching either takes the edge into the matching or it does not.

TABLE ENTRIES: We can define a DP table similarly to our second approach for Maximum Independent Set on trees. The table entry $M[node, can_take]$ is defined as

- $M[node, 1]$: the maximum matching size in the subtree rooted at $node$.
- $M[node, 0]$: the maximum matching size in the subtree rooted at $node$ that does **not** take any edge incident to $node$.

RECURRENCE:

$$M[node, 0] = M[node.left, 1] + M[node.right, 1]$$

$$M[node, 1] = \max \begin{cases} M[node, 0] & \text{(do not take any edge to a child)} \\ 1 + M[node.left, 0] + M[node.right, 1] & \text{(take edge to left child)} \\ 1 + M[node.right, 0] + M[node.left, 1] & \text{(take edge to right child)} \end{cases}$$

We end this section with a formal proof of correctness for the recurrence.

CORRECTNESS:

$M[node, 0]$: Every matching not taking an edge from $node$ to a child is a union of matchings in the left and right subtrees. Similarly, any such union of matchings forms a valid matching for the subtree rooted at $node$, without taking an edge incident to $node$. To form an optimal solution, one might as well take the optimal solutions for the left and right children subtrees.

$M[node, 1]$: An optimal matching either takes an edge to a child, or it does not. If it does not, the best solution is just the one for $M[node, 0]$. Otherwise, the matching must take exactly one edge, either to the left child or to the right child. Without loss of generality, say the left edge is taken, then any solution must not take any other edge for the left child. Consequently, in this case, the best solution conditioned on taking the $node$ -to-left-child edge is $1 + M[node.left, 0] + M[node.right, 1]$. A symmetric argument argues the right child case, showing the correctness of the recurrence.

4.8 Shortest Paths on Graphs

SETTING: We are given a directed, weighted n -vertex graph $G = (V, E)$ where each edge e has weight w_e . These weights may be negative. Our goal is to compute the shortest paths between vertices.

There are two main variations of this problem:

- **Single-Source:** Given a specific vertex u , compute the shortest path from u to all other vertices v .
- **All-Pairs:** For all pairs of vertices u and v , compute the shortest path from u to v .

4.8.1 Single-Source Shortest Path: Bellman-Ford

Our goal here is to derive the Bellman-Ford algorithm using dynamic programming. This will, interestingly, require different intuition from the basic “either/or” case analysis we have been using in previous examples.

To start, we need to define the DP table. We could try the approach we had on trees, and define $D[v]$ (D for distance) as

$$D[v] = \text{shortest path length from } u \text{ to } v$$

The subproblem structure is implicit in what the shortest paths are: every path is a sequence of hops, and a vertex with fewer hops in a shortest path gives a subproblem for vertices with more hops further along the path. But this is circular, since the shortest paths are exactly what we are trying to compute!

Given this hop intuition, we form another DP table that tracks the hop count to define subproblems.

TABLE ENTRIES: Let $D[v, k]$ be the shortest path distance from u to v using at most k hops. We will also insert self-loops of weight 0 for all vertices, and equivalently define $D[v, k]$ be the shortest path distance from u to v using *exactly* k hops, which is simpler.

RECURRENCE: Defining $w_{x \rightarrow v}$ to be ∞ whenever the edge $x \rightarrow v$ does not exist, the following is the recurrence for the $D[v, k]$ table.

$$D[v, 0] = \begin{cases} \infty & \text{if } v \neq u \\ 0 & \text{if } v = u \end{cases}$$

$$D[v, k] = \min \left(D[v, k-1], \min_{x \neq v} (D[x, k-1] + w_{x \rightarrow v}) \right)$$

We should be careful that defining a table also requires defining the bounds on the table indices. Here, this means answering how large k can be: the maximum number of edges on any shortest path is $|V| - 1$, at least, unless the graph has a negative cycle (a cycle whose edge weights sum to below 0), in which case the notion of shortest paths is not even defined. More on negative cycles later.

PSEUDOCODE: Here is an implementation.

Algorithm 4.4. Bellman-Ford Algorithm

1. For all k , initialize $D[v, k] \leftarrow \infty$ for all $v \neq u$ and $D[u, k] \leftarrow 0$.
 2. for $k = 1$ to $n - 1$:
 - (a) for each edge $x \rightarrow v$: $D[v, k] \leftarrow \min(D[v, k], w_{x \rightarrow v} + D[x, k-1])$
-

Here, in this implementation, we only enumerate edges present in the graph instead of all pairs (x, v) . Thus, only $O(|E|)$ computation is done in each outer loop, giving a total of $O(|V||E|)$ runtime.

We can, again, optimize the space usage of the Bellman-Ford algorithm. As in prior examples, we only need the values at index $k - 1$ to compute the values at index k .

In fact, it is possible to drop the hop-count index altogether and just keep updating a single D array, though the correctness analysis is trickier and omitted here.

Algorithm 4.5. Space-Optimized Bellman-Ford Algorithm

1. Initialize $D[v] \leftarrow \infty$ for all $v \neq u$ and $D[u] \leftarrow 0$.
 2. for $k = 1$ to $n - 1$:
 - (a) for each edge $x \rightarrow v$: $D[v] \leftarrow \min(D[v], w_{x \rightarrow v} + D[x])$
-

4.8.2 Negative Cycles

However, as mentioned above, weighted graphs may contain negative cycles, and the notion of shortest paths (absent constraints on the number of hops) might not even be well-defined. It turns out that Bellman-Ford can even be used to detect the existence of negative cycles reachable from the source vertex u , by running the outer loop one more time. To see this, we state and prove the following claim.

Claim 4.2 *There is a negative cycle reachable from u if and only if there exists some vertex v such that $D[v, n] < D[v, n - 1]$.*

Proof. $D[v, n] < D[v, n - 1] \Rightarrow$ existence of negative cycle reachable from u : Let $u \rightarrow a_1 \rightarrow \dots \rightarrow a_n = v$ be an n -hop path of minimum weight for $D[v, n]$. By the pigeonhole principle, there must exist some pair of indices $i < j$ such that $a_i = a_j$ as the path involves $n + 1$ vertices.

Now consider the following $n - 1$ hop path $u \rightarrow a_1 \rightarrow \dots \rightarrow a_i \rightarrow a_{j+1} \rightarrow \dots \rightarrow v \rightarrow v \rightarrow \dots \rightarrow v$ where the padding by v makes the path exactly $n - 1$ hops. The weight of this path upper bounds $D[v, n - 1]$, which in turn is strictly greater than $D[v, n]$ that is equal to the weight of the previous path.

The weight of the cycle $a_i \rightarrow \dots \rightarrow a_j = a_i$ is equal to the difference between the weights of the two paths, and is hence strictly negative.

(For all v , $D[v, n] \geq D[v, n - 1]$) $\Rightarrow$ there are no negative cycles reachable from u : Consider any arbitrary cycle $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_1$ that is reachable from u . For any i , we know that

$$D[a_{i+1}, n - 1] \leq D[a_{i+1}, n] \leq D[a_i, n - 1] + w_{a_i \rightarrow a_{i+1}}$$

Summing this inequality over all edges in the cycle yields:

$$\sum_i D[a_{i+1}, n - 1] \leq \sum_i D[a_i, n - 1] + \sum_i w_{a_i \rightarrow a_{i+1}}$$

The distance sum on both side cancels out, leaving $\sum_i w_{a_i \rightarrow a_{i+1}} \geq 0$. ■

Algorithmically, we can simply run one additional outer loop (the n -th iteration) to check if any distances are updated. If they are, a negative cycle exists.

4.8.3 All-Pairs Shortest Path

If we simply run Bellman-Ford using every vertex as a source, we can compute all-pairs shortest path in $O(|V|^2|E|)$ time. However, a graph can have as many as $O(|V|^2)$ edges, making this basic approach require $O(|V|^4)$ time in the worst case.

The Floyd-Warshall algorithm improves this to $O(|V|^3)$ time. It crucially assumes that there are no negative cycles in the graph.

The subproblem structure here is not necessarily obvious.

TABLE ENTRIES: The DP table has entries $D[i, j, k]$, where i, j and k are all vertices. The entry is defined as

$D[i, j, k]$ = the shortest path length from i to j ,
where all intermediate vertices on the path are constrained to lie in $[1..k]$

RECURRENCE: To derive the recurrence, the “either or” observation is that a shortest path for $D[i, j, k]$ must either use vertex k exactly once or it does not. If k is not used, then $D[i, j, k-1]$ is the best possible path. If k is used, then the path must go from i to k using vertices in $[1..(k-1)]$ then from k to j using vertices in $[1..(k-1)]$. This yields the recurrence

$$D[i, j, k] = \min(D[i, j, k-1], D[i, k, k-1] + D[k, j, k-1])$$

$$D[i, j, 0] = w_{i \rightarrow j} \quad (\text{which may be } \infty \text{ for non-existent edges})$$

PSEUDOCODE: The algorithm is implemented as three nested loops. Here it is crucial that k is the outer loop, but the nesting order of the inner loops does not matter.

Algorithm 4.6. Floyd-Warshall Algorithm

1. for $k = 1$ to n :
 - (a) for $i = 1$ to n :
 - i. for $j = 1$ to n : $D[i, j, k] \leftarrow \min(D[i, j, k-1], D[i, k, k-1] + D[k, j, k-1])$
-

Similar to Bellman-Ford, we can optimize the space complexity by dropping the k index entirely, updating the 2D array in place:

$$D[i, j] = \min(D[i, j], D[i, k] + D[k, j])$$

Greedy Algorithms

In this chapter, we introduce the Greedy Algorithm paradigm.

A *greedy algorithm* constructs a (partial) solution iteratively via a sequence of short-sighted decisions or choices, with the hope that it works out at the end.

The above quote/informal definition sounds tentative and hesitant, and indeed that is the intention. It is crucial to understand that *most* greedy algorithms are wrong, however tempting it may be to use one.

Having said that, in the case of optimization problems, greedy algorithms are frequently a decent strategy for *approximation* algorithms, namely algorithms that output solutions that are provably not too much worse than the optimum. We will see some examples in Chapter 6, but the detailed study is generally rather out of scope for this course.

5.1 Motivating Example: Fractional Knapsack

We start the discussion using the motivating example problem of Fractional Knapsack.

SETTING: We are given a knapsack with a maximum weight capacity $C \geq 0$, and a list of n completely divisible items (think liquids). Each item $i \in \{1, \dots, n\}$ has a specific weight $w_i \geq 0$ and a total reward $r_i \geq 0$. We are allowed to take any fractional amount $\rho_i \in [0, 1]$ of item i , which contributes a weight of $\rho_i w_i$ to the knapsack and yields a reward of $\rho_i r_i$. The goal is to maximize the total reward subject to the capacity constraint:

$$\sum_{i=1}^n \rho_i w_i \leq C$$

What are some possible greedy strategies to select items? We might consider greedily filling the knapsack with:

1. The item with the smallest weight w_i .
2. The item with the biggest reward r_i .
3. The item with the highest reward density r_i/w_i .

The first two strategies are incorrect. To see why, consider a knapsack with capacity $C = 100$ and the following three items:

Item	Weight (w_i)	Reward (r_i)	Density (r_i/w_i)
1	100	100	1
2	10	60	6
3	1	0.1	0.1

- **Smallest w_i fails:** Greedily taking the smallest weight first results in taking Item 3, then Item 2, and finally 89/100 of Item 1, yielding a total reward of $0.1 + 60 + 89 = 149.1$.
- **Biggest r_i fails:** Greedily taking the biggest reward first results in taking Item 1, exhausting the knapsack capacity immediately for a total reward of 100.

The optimal approach, instead, is to select based on the highest reward density r_i/w_i . In the above example, the approach would fully take Item 2 and then 90/100 of Item 1, achieving a maximum possible reward of $60 + 90 = 150$.

More formally, the following is an implementation of this correct greedy algorithm.

Algorithm 5.1. Fractional Knapsack

1. Sort items in decreasing order of reward density r_i/w_i .
 2. $total_weight \leftarrow 0$
 3. for $i \leftarrow 1$ to n :
 - (a) if $total_weight + w_i \leq C$:
 - i. Take $\rho_i \leftarrow 1$
 - ii. $total_weight \leftarrow total_weight + w_i$
 - (b) else:
 - i. Take $\rho_i \leftarrow \max\left(\frac{C - total_weight}{w_i}, 0\right)$
 - ii. $total_weight \leftarrow C$
-

5.1.1 The General Proof Framework

But why is Algorithm 5.1 actually correct? How do we *prove* that it is correct?

A commonly-held but **wrong** intuition for the correctness of a greedy algorithm is that it makes a series of “locally optimal choices” to yield a “globally optimal” solution.

But there are two important (and in hindsight glaringly obvious) issues with this intuition. First, this intuition only discusses optimization problems, but greedy algorithms (as we will see) can also apply to feasibility/satisfiability problems where the goal is to find *any* solution satisfying a set of constraints. Second, even for optimization problems, think about how we would use induction to prove correctness, where the induction variable is, naturally, the number of choices and commitments made by the

greedy algorithm. During the execution of the algorithm, most of the time there will be some partially-constructed solution, which does not necessarily have a “solution value” or “solution quality” associated with it, unlike a fully constructed solution. Because of this, it is quite unclear how one would formalize the way these intermediate solutions relate to “global optimality”, whatever that means.

Instead, the **correct** intuition for greedy algorithms is that:

For every choice the greedy algorithm makes, it did not mess up.

While this intuition sounds informal, we can in fact capture it using induction, with the following proof framework:

- **Induction Variable:** The number of choices made in the algorithm/partial solution.
- **Induction Hypothesis:** For the partial solution constructed from the first i choices of the algorithm, there **exists** an optimal (or feasible) solution that is “compatible” with this partial solution.

The notion of “compatibility” depends on the problem at hand, and it is critical for you as the algorithm designer and analyst to explicitly specify what it means exactly, much like how it is your responsibility to determine and specify what “correct” means.

5.1.2 Proving Correctness for Fractional Knapsack

Let us now apply the above framework to prove the correctness of [Algorithm 5.1](#).

INDUCTION HYPOTHESIS: Let $\rho_1, \dots, \rho_i$ be the partial solution constructed at the end of the i^{th} loop iteration. Then, there exists an¹ optimal solution $\rho_1^{(opt)}, \dots, \rho_n^{(opt)}$ such that $\rho_1 = \rho_1^{(opt)}, \dots, \rho_i = \rho_i^{(opt)}$.

For this specific problem and algorithm, the notion of “compatibility” is simple, that an optimal solution includes the choices that the algorithm made.

BASE CASE ($i = 0$): The partial solution is empty, so it trivially agrees with any optimal solution on the first 0 choices.

INDUCTION STEP: Assume there is an optimal solution such that $\rho_1 = \rho_1^{(opt)}, \dots, \rho_i = \rho_i^{(opt)}$. We need to show that there is also an optimal solution compatible additionally with ρ_{i+1} chosen by the algorithm. Note, carefully, that we are allowed to construct a *new* optimal solution as long as it does remain optimal.

Consider the choice ρ_{i+1} made in the $i + 1^{\text{st}}$ loop iteration. In either case of the loop iteration (whether $\rho_{i+1} = 1$ or $\rho_{i+1} < 1$), it must be true that $\rho_{i+1} \geq \rho_{i+1}^{(opt)}$.

¹Note that we use the word “an” not “the”, since there might be multiple optimal solutions, and indeed this is something we consider in the proof.

If $\rho_{i+1} = \rho_{i+1}^{(opt)}$, then the assumed optimal solution is already compatible with choice $i + 1$, and we are done.

If they are not equal, then we have $\rho_{i+1} > \rho_{i+1}^{(opt)}$. Thus, there will be at least $(\rho_{i+1} - \rho_{i+1}^{(opt)})w_{i+1}$ weight taken by the $\rho^{(opt)}$ solution in items $i+2$ through n ; otherwise, there is unused capacity in that solution, and we might as well increase $\rho_{i+1}^{(opt)}$ to use up the capacity.

We now construct a new optimal solution $\rho'_1, \dots, \rho'_n$ by taking $(\rho_{i+1} - \rho_{i+1}^{(opt)})w_{i+1}$ weight away from items $i + 2$ through n in the $\rho^{(opt)}$ solution, and moving that weight back to item $i + 1$.

Since the algorithm sorted the items by density, item $i + 1$ has reward density at least that of any item from $i + 2$ to n . Therefore, this new solution ρ' has a total reward that is no worse than $\rho^{(opt)}$, implying that ρ' must also be an optimal solution. By construction, ρ' is equal to the greedy algorithm's choices up to item $i + 1$: $\rho'_1 = \rho_1, \dots, \rho'_{i+1} = \rho_{i+1}$. This completes the induction step.

The above is an instance of a **swapping argument** that is common for analyzing greedy algorithms.

WRAP UP: At the end of the n^{th} loop iteration, the induction hypothesis dictates that there exists an optimal solution $\rho^{(opt)}$ such that $\rho_1 = \rho_1^{(opt)}, \dots, \rho_n = \rho_n^{(opt)}$. This implies the algorithm's solution ρ is optimal.

5.2 Choose Your Team Tournament

We now look at a *feasibility* problem, as opposed to an optimization problem, where being greedy is a correct approach.

SETTING: There are two teams, Team A and Team B, each consisting of n members. The members have power levels represented in arrays $A[1..n]$ and $B[1..n]$, which are not necessarily sorted. Each member of Team A will compete in a 1-on-1 match with exactly one member of Team B, where a Team A member wins if and only if their power level is at least the Team B opponent's. Supposing that Team A chooses the matching, is it possible for Team A to win all n matches?

To derive an algorithm, consider trying to find a Team A member to match up with $B[1]$. Which Team A member would be a choice that you *might as well* make? We need to make sure to win that match, so the choice must be a member whose power level is at least that of $B[1]$. But which one should we pick? In this case, we might as well pick the weakest such Team A member. And this process can be repeated until all team members are matched, yielding the following algorithm.

Algorithm 5.2. Choose Your Team Tournament

1. for $i \leftarrow 1$ to n :
 - (a) Find the weakest member $A[j]$ such that $A[j] \geq B[i]$.
 - (b) If such an element exists, match $A[j]$ with $B[i]$ and remove $A[j]$ from consideration.
 - (c) If no such element exists, immediately output “no”.
2. If the loop completes successfully, output “yes” (and the constructed matching).

RUNTIME ANALYSIS: This can be implemented in $O(n \log n)$ time by judiciously using a binary search tree to store elements of Team A, but we omit the details here.

5.2.1 Correctness

We want to show that **Algorithm 5.2** outputs “yes” if and only if there exists a winning matching. If the algorithm terminates and outputs “yes”, then by the algorithm design it explicitly constructed a valid winning matching. It remains to show the converse: If Team A can win, then the algorithm will successfully find a winning matching.

INDUCTION HYPOTHESIS: If Team A can win, then there exists a winning matching that agrees with the matches made by the algorithm in the first i loop iterations.

BASE CASE ($i = 0$): Trivial, as 0 matches have been made.

INDUCTION STEP: Assume there exists a winning matching M that agrees with the algorithm’s first i match choices. Now, suppose that in the $i + 1^{\text{st}}$ iteration, the algorithm matches $B[i + 1]$ with $A[j]$.

If M also matches $B[i + 1]$ with $A[j]$, then M already agrees with the first $i + 1$ choices, and we are done.

Otherwise, M matches $B[i + 1]$ with some different element $A[j']$, and M must match $A[j]$ with some element $B[i']$ where $i' > i + 1$ (i' cannot be $\leq i$ since M matches $B[1..i]$ exactly like the algorithm does, and by assumption $i' \neq i + 1$ either).

Note that by the algorithm’s greedy criterion, we have $A[j'] \geq A[j]$. Furthermore, since M is a valid winning matching, we know that $A[j'] \geq B[i + 1]$ and $A[j] \geq B[i']$.

We can now construct a new matching M' by swapping these specific assignments: match $A[j]$ with $B[i + 1]$, and match $A[j']$ with $B[i']$. We must verify that this new matching is still winning:

- $A[j] \geq B[i + 1]$: The algorithm explicitly checked this for $A[j]$.
- $A[j'] \geq B[i']$: Since $A[j'] \geq A[j]$ and $A[j] \geq B[i']$, we have $A[j'] \geq B[i']$.

Thus, M' is still a winning matching, and it now agrees with the algorithm on the first $i + 1$ choices. This completes the induction step.

WRAP UP: If Team A can win, then by the end of the n^{th} loop iteration, the induction hypothesis guarantees there exists a winning matching that agrees with the algorithm on all n choices, meaning that the loop will have found a winning matching. Thus the algorithm outputs “yes”.

5.3 Data Compression and Huffman Coding

In this section, we turn our attention to the problem of encoding messages, which are strings of symbols, into binary bit strings. The primary motivations for studying this problem are data compression (specifically, lossless compression) and data transmission (high-voltage vs low-voltage, binary bits). The provably optimal algorithm, the classic Huffman Coding, turns out to be a greedy strategy.

5.3.1 Problem Setting: Binary and Prefix-Free Codes

Before discussing the algorithm, we first need to introduce the coding setting and a number of technical terms.

- Σ is an *alphabet*, meaning the set of possible message symbols. For example, the set of all English letters is an alphabet. The set $\{0, 1\}$ is the binary alphabet.
- $\{0, 1\}^*$ is the set of all (finite-length) binary bit strings. The $*$ denotes “any possible integer length”.
- A *binary code* is an injective function mapping $\Sigma \rightarrow \{0, 1\}^*$, meaning that the code maps every possible symbol into a binary bit string. The binary bit string is also called a *codeword*.

The simplest approach to design a code is to have a *fixed-length code*, where every codeword has the exact same length for every symbol. To encode an alphabet of size $|\Sigma|$, the minimum required length for a fixed-length code is $\lceil \log_2 |\Sigma| \rceil$. On the other hand, a *variable-length code* has no such restriction. For example, consider the alphabet $\Sigma = \{a, b, c, d\}$ and the code $C = \{a \mapsto 0, b \mapsto 10, c \mapsto 110, d \mapsto 111\}$.

Why would we use a variable-length code, say, over the fixed-length code (00, 01, 10, 11)? Suppose we know that the symbol a is much more likely to appear than other symbols, at higher frequency. In such setting, the variable-length code will have much shorter “average” code length or “expected” code length. We will formalize this later.

With variable-length codes, we must be careful to avoid ambiguities during decoding. One way to ensure unique decoding is to use a *prefix-free* code: a (binary) code C such that for every pair of distinct symbols $\sigma, \sigma' \in \Sigma$, the codeword $C(\sigma)$ is not a prefix of the codeword $C(\sigma')$.

Note that fixed-length codes are always trivially prefix-free. The variable-length code C provided in the example above is also prefix-free.

5.3.2 The Optimal Prefix-Free Binary Code Problem

Let us now formalize the optimization problem for code design. Suppose we are given an alphabet Σ and a probability P_σ associated with each symbol $\sigma \in \Sigma$. Our goal is to

compute a prefix-free binary code C that minimizes the expected code length:

$$\min \sum_{\sigma \in \Sigma} P_{\sigma} |C(\sigma)|$$

where $|C(\sigma)|$ denotes the length of the codeword $C(\sigma)$.

Before designing an algorithm, we make three critical structural observations.

1. Prefix-free binary codes can be represented as binary trees, with symbols $\sigma \in \Sigma$ as leaf nodes, and the edges are labeled with 0 and 1 (ensuring no shared label exists for both edges out of any parent node).

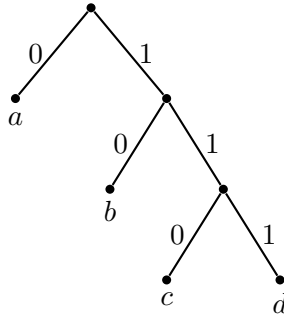


Figure 5.1. A prefix-free binary code represented as a binary tree. The symbols are located at the leaves.

2. The code length for a symbol σ is exactly the depth of its corresponding leaf node in the tree.
3. Only the tree structure itself matters for the optimization objective; any valid 0/1 labeling of the edges yields the exact same expected code length.

5.3.3 The Huffman Coding Algorithm

Huffman Coding is a greedy algorithm designed to compute an optimal prefix-free binary code. The algorithm maintains a list of subtrees, where each subtree encodes a disjoint subset of the symbols. It repeatedly merges pairs of subtrees, until a single tree remains: the greedy rule is to always pair two subtrees with the smallest total probabilities.

Algorithm 5.3. Huffman Coding

-
1. for $\sigma \in \Sigma$:
 - (a) $T_\sigma \leftarrow$ a single leaf node “ σ ”
 - (b) $P(T_\sigma) \leftarrow P_\sigma$
 2. $List \leftarrow \{T_\sigma\}_{\sigma \in \Sigma}$ // Initialize the set of subtrees
 3. while $List$ size > 1 :
 - (a) $T_1, T_2 \leftarrow$ the subtrees with the smallest and second smallest $P(T)$ in $List$
 - (b) $T_{\text{merge}} \leftarrow \bigwedge_{T_1 \ T_2}$; $P(T_{\text{merge}}) \leftarrow P(T_1) + P(T_2)$
 - (c) Remove T_1, T_2 from $List$
 - (d) Add T_{merge} back to $List$
 4. Return the final single tree from $List$
-

5.3.4 Correctness of Huffman Coding

We will prove the correctness of the Huffman coding algorithm using the proof framework we saw earlier.

INDUCTION HYPOTHESIS: At the end of the i^{th} iteration, there exists an optimal code tree such that all trees currently in $List$ are subtrees of this optimal tree.

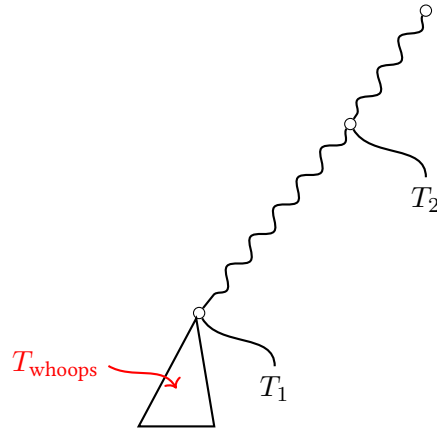
BASE CASE: Before any iterations ($i = 0$), the list consists merely of individual leaves for each symbol in Σ . By definition, any valid code tree must contain these individual leaves, so the base case trivially holds.

INDUCTION STEP: Let OPT_i be the optimal code tree guaranteed by the hypothesis, which contains all subtrees in $List$ at the beginning of the $i + 1^{\text{st}}$ iteration. During the $i + 1^{\text{st}}$ iteration, suppose the algorithm merges T_1 and T_2 into T_{merge} . If T_{merge} is already a subtree of OPT_i (meaning T_1 and T_2 are siblings in OPT_i), then we are done.

Otherwise, we assume without loss of generality that T_1 is positioned deeper (no shallower) in OPT_i than T_2 . Since T_1 is not paired with T_2 , its sibling in OPT_i must be some other subtree, which we will call T_{whoops} .

Notice that T_{whoops} must contain at least one subtree in $List$, and it cannot be T_1 or T_2 . By the greedy criterion, T_1 and T_2 have the smallest probabilities in the list, so $P(T_{\text{whoops}}) \geq P(T_2)$.

We modify OPT_i by swapping the positions of T_{whoops} and T_2 . As T_1 is no shallower than T_2 , swapping T_2 into T_{whoops} 's deeper position and pulling T_{whoops} up does not increase the overall expected code length. In other words, the new tree has at most the expected code length as OPT_i , meaning the new tree is also optimal. Furthermore, this new optimal tree has $\bigwedge_{T_1 \ T_2}$ as a subtree, completing the induction step.



5.3.5 Huffman Code Length and Discrete Entropy

Definition 5.1 (Discrete Entropy) Consider a discrete random variable X over a domain $\mathcal{X}$. The discrete entropy of X is defined as:

$$H(X) = - \sum_{x \in \mathcal{X}} p_x \log p_x$$

This definition is relevant due to Claude Shannon's Source Coding Theorem.

The proof of this theorem is graduate-level and requires semi-advanced intuition for probability. However, we can state a concrete upper bound for the Huffman Code's expected length relative to the Shannon entropy limit.

$$\mathbb{E}_X[|C(X)|] \leq H(X) + 1$$

The proof is also challenging, and I am unaware of any direct analysis of Huffman Code showing this bound. Instead, typical proofs show that a different prefix-free coding scheme, called the Shannon-Fano code, achieves this $H(X) + 1$ upper bound. Huffman Code's performance then follows from its optimality among all prefix-free codes.

5.4 Minimum Spanning Trees

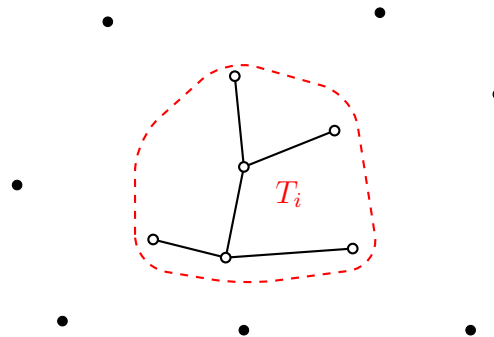
SETTING: Given a connected, weighted, undirected n -vertex graph $G = (V, E)$, the goal is to compute a spanning tree with the minimum total edge weight. Recall that a spanning tree is a tree connecting all vertices, hence with $n - 1$ edges.

There are two standard greedy algorithms for this problem: Prim's algorithm and Kruskal's algorithm. For this course, we only have time to cover Prim's algorithm.

5.4.1 Prim's Algorithm

The core idea is to grow a spanning tree outward from a single vertex, adding one edge at a time. Let us try deriving the algorithm.

Consider some intermediate stage where we have a partial tree T_i . What does any spanning tree need to do? It needs to connect T_i with *some* vertex outside of T_i .



The key observation is that we *may as well* take the minimum-weight edge between T_i and $V \setminus T_i$.

Algorithm 5.4. Prim's Algorithm

-
1. $Connected \leftarrow \{s\}$ // s arbitrarily chosen
 2. $T \leftarrow \emptyset$ // empty tree, no edges
 3. while there exists edge between $Connected$ and $V \setminus Connected$:
 - (a) Let $(u, v) = \arg \min_{\substack{u \in Connected \\ v \notin Connected \\ (u, v) \in E}} w_{(u, v)}$
 - (b) Add (u, v) to T
 - (c) Add v to $Connected$
 4. return T
-

5.4.2 Correctness

We can again prove the correctness of Prim's algorithm using the framework introduced earlier.

INDUCTION HYPOTHESIS: At the end of the i^{th} iteration, there exists a minimum spanning tree T_i^{opt} such that T_i^{opt} contains all edges in T .

BASE CASE: $i = 0$, trivial.

INDUCTION STEP: Assume the induction hypothesis for iteration i , so there is a minimum spanning tree T_i^{opt} containing all of T_i , where T_i is the state of T at end of iteration i .

Call (u, v) the new edge added in iteration $i + 1$, with $u \in T_i$ and $v \notin T_i$. If $(u, v) \in T_i^{opt}$, then we are done (explicitly, let $T_{i+1}^{opt} = T_i^{opt}$). Otherwise, $(u, v) \notin T_i^{opt}$, so $T_i^{opt} \cup \{(u, v)\}$ has a cycle.

At this point, we need to identify some component (really, edge) in T_i^{opt} to swap with (u, v) . Now the claim is that there must exist some edge $(a, b) \neq (u, v)$ on that cycle where $a \in T_i$ and $b \notin T_i$. Why? Consider walking along the cycle from u , starting from $u \rightarrow v$. This edge goes from T_i to $V \setminus T_i$. But eventually, along the walk, we end up back at u which is in T_i . Therefore, along the way there must be some $(a, b) \neq (u, v)$ in the cycle such that $a \in T_i$ and $b \notin T_i$.

To construct T_{i+1}^{opt} , we swap (a, b) with (u, v) , namely

$$T_{i+1}^{opt} = T_i^{opt} \cup \{(u, v)\} \setminus \{(a, b)\}$$

By the greedy criterion of the algorithm, we know that $w_{(a, b)} \geq w_{(u, v)}$, implying that $w(T_{i+1}^{opt}) \leq w(T_i^{opt})$. Furthermore, T_{i+1}^{opt} still has $n - 1$ edges and connects all vertices, hence is a spanning tree. Therefore, T_{i+1}^{opt} is another minimum spanning tree, which by construction contains T_{i+1} as desired.

WRAP UP: Upon termination, T connects all vertices reachable from s , and there exists a minimum spanning tree containing all of T . So T is a minimum spanning tree.

5.4.3 Implementation

The algorithmically challenging part is finding the minimum outgoing edge from T efficiently. We can use a heap data structure to achieve this goal.

Heaps support the following operations:

- **INSERT**($x = (\text{KEY}, \text{VALUE})$): adds x into the data structure.
- **EXTRACTMIN**(): returns an element with the minimum key and deletes it from the data structure.
- **DELETE**(**POINTER TO ELEMENT** x): deletes a specific element x from the data structure.

FACT: There is a heap implementation such that all operations take $O(\log n)$ time, where n is the number of elements in the heap at the time.

For Prim's algorithm, we can use such a heap to store vertices to decide which one to grow the tree to. See [Algorithm 5.5](#), which runs in $O((|V| + |E|) \log |V|)$ time.

Algorithm 5.5. Prim's Algorithm via Heap

1. $Connected \leftarrow \{s\}; T \leftarrow \emptyset; H \leftarrow \text{empty heap}$
 2. $Pointers \leftarrow$ array indexed by V , initialized to NULL
 3. for every $v \neq s$:
 - (a) if $(s, v) \in E$ then $key(v) \leftarrow w_{(s,v)}$ else $key(v) \leftarrow \infty$
 - (b) $edge(v) \leftarrow (s, v)$
 - (c) Create $(key(v), (v, edge(v)))$ with pointer p_v , add to H and store p_v in $Pointers[v]$
 4. while H is non-empty:
 - (a) $(key(u), (u, edge(u))) \leftarrow \text{EXTRACTMIN}(H)$
 - (b) Add u to $Connected$, add $edge(u)$ to T
 - (c) for each edge (u, v) where $v \notin Connected$:
 - i. if $key(v) > w_{(u,v)}$:
 - A. DELETE v from H using $Pointers[v]$
 - B. $key(v) \leftarrow w_{(u,v)}$; $edge(v) \leftarrow (u, v)$
 - C. Create entry $(key(v), (v, edge(v)))$ for v in H with pointer p_v , update $Pointers[v]$
 5. return T
-

To reason about data structures (and this is the only small part of the course where we can afford the time to discuss how to reason about them), it is crucial to specify an invariant that keeps holding true no matter what sequence of allowable operations is made to the structure.

INVARIANT: At the end of every loop iteration, the heap H contains all vertices $v \in V \setminus \text{Connected}$ with key $\min_{u \in \text{Connected}} w_{(u,v)}$.

5.5 Dijkstra's Algorithm

We now return to the single-source shortest path problem. Previously, in the dynamic programming chapter, we covered the Bellman-Ford algorithm which runs in $O(|V||E|)$ time, while allowing negative weights and even detects the existence of negative cycles reachable from the source vertex. Now we introduce Dijkstra's algorithm, another classic greedy algorithm, which by contrast can be implemented to run in $O((|V|+|E|) \log |V|)$ time, but it assumes non-negative edge weights. The structure of Dijkstra's algorithm, as we will see, is remarkably similar to Prim's algorithm.

NOTE: Our presentation of Dijkstra's algorithm will be somewhat different from standard sources, textbook or otherwise. Standard sources tend to describe and analyze Prim's and Dijkstra's algorithms significantly differently from each other, even though they are fundamentally very similar. The reformulation in this section is designed to unify their treatment under the same analysis framework introduced at the beginning of the chapter.

5.5.1 Shortest Path Tree

Prim's algorithm computes a minimum spanning tree, and we need an analogous notion for the shortest path problem in order to derive Dijkstra's algorithm.

Definition 5.4 (Shortest path tree) A directed tree T is a *shortest path tree* with respect to a directed graph G and source vertex s if:

1. T contains all reachable vertices from s ,
2. T is a subgraph of G , and
3. For all such vertices v , the path $s \rightsquigarrow_T v$ in T is a shortest path in G .

A priori (and without relying on a shortest path algorithm), it might not even be obvious that shortest path trees should exist, so we first show that here, before deriving Dijkstra's algorithm to compute one.

EXISTENCE: For any vertex v reachable from s , choose any shortest path $s \rightsquigarrow_G v$ with the fewest hops. Let $prev(v)$ be the vertex before v on this path, and let $hop(v)$ be the number of hops on the chosen path.

Observe that $hop(prev(v)) = hop(v) - 1$ (this is left as an exercise to prove for the reader).

Finally, we can construct a shortest path tree hop-by-hop, adding edges $prev(v) \rightarrow v$ for every $v \neq s$. By construction, there is one edge added per reachable vertex, with no cycles, so the graph is indeed a tree. We leave this as an exercise to show that indeed the tree is a shortest path tree, by induction on the number of hops.

5.5.2 Algorithm

The idea of Dijkstra's algorithm is very similar to Prim's, to grow a shortest path tree from source s . Consider again, like in our discussion for Prim's algorithm, some intermediate stage where we have a partial tree T_i (which, eventually by induction, will be guaranteed to be a subtree of some shortest path tree). A shortest path tree, being a spanning tree, again needs to connect a vertex in T_i to some vertex outside of T_i . Which edge should we pick at the boundary of T_i to grow it into T_{i+1} ? Dijkstra's algorithm greedily picks the edge $u \rightarrow v$ so that the shortest path from $s \rightsquigarrow_{T_i} u$ plus $w_{u \rightarrow v}$ is the shortest, as follows.

Algorithm 5.6. Dijkstra's Algorithm

1. $Connected \leftarrow \{s\}$
 2. $T \leftarrow \emptyset$
 3. while there exists an edge from $Connected$ to $V \setminus Connected$:
 - (a) Find $(u \rightarrow v) = \arg \min_{\substack{u \in Connected \\ v \notin Connected \\ (u \rightarrow v) \in E}} dist_T(s, u) + w_{u \rightarrow v}$
 - (b) Add $u \rightarrow v$ to T and v to $Connected$.
 4. return T
-

EXERCISE: Figure out how to implement this in $O((|V| + |E|) \log |V|)$ time using the heap data structure.

5.5.3 Correctness

Given the similarity to Prim's algorithm, we can prove the correctness of Dijkstra's analogously as well.

INDUCTION HYPOTHESIS: At the end of every iteration i , T_i (the state of T after iteration i) is such that

1. $Connected_i$ (the state of $Connected$ after iteration i) is the set of v reachable from s in T_i , and the number of edges in T_i is exactly $|Connected_i| - 1$.
2. There exists a shortest path tree T_i^{opt} containing all edges in T_i .

A corollary of the second point is that $dist_{T_i}(s, v) = dist_{T_i^{opt}}(s, v)$. The $\geq$ direction is because T_i^{opt} is a shortest path tree, and the $\leq$ direction is because T_i^{opt} contains all of T_i and so the paths of T_i are shortest paths.

INDUCTION STEP:

1. Condition 1 is trivial. Every v we add is reachable from s .
2. Suppose T_i^{opt} is a shortest path tree containing all of T_i by induction.
Let $u \in T_i \rightarrow v \notin T_i$ be the edge chosen in iteration $i + 1$.

If $u \rightarrow v$ is already in T_i^{opt} , then let $T_{i+1}^{opt} = T_i^{opt}$ and Condition 2 holds.

Otherwise, suppose $u \rightarrow v$ is not in T_i^{opt} . Then in T_i^{opt} , the path $s \rightsquigarrow v$ must contain some edge $a \rightarrow b$ where $a \in T_i$ and $b \notin T_i$, and by assumption $a \rightarrow b$ cannot be $u \rightarrow v$. So the path looks like $s \rightsquigarrow a \rightarrow b \rightsquigarrow prev(v) \rightarrow v$.

Observe that

$$\begin{aligned} dist_{T_i^{opt}}(s, v) &\geq dist_{T_i^{opt}}(s, a) + w_{a \rightarrow b} && \text{(No negative weight edges)} \\ &= dist_{T_i}(s, a) + w_{a \rightarrow b} \\ &\geq dist_{T_i}(s, u) + w_{u \rightarrow v} && \text{(Greedy criterion)} \end{aligned}$$

With this, we construct T_{i+1}^{opt} by removing $prev(v) \rightarrow v$ (which cannot be in T_i) and adding $u \rightarrow v$. Reachability is preserved, and all the path distances are no longer than in T_i^{opt} . Thus T_{i+1}^{opt} satisfies Condition 2.

WRAP UP: At termination, there is some shortest path tree T^{opt} containing T , and they have the same number of vertices, meaning that the edge count is the same ($|Connected| - 1$). Thus, they are the same tree.

Reductions, NP-Hardness and Approximation Algorithms

In this last chapter, we develop the algorithmic notion of *reductions*, and use it to motivate how we can reason about the *hardness* of a computational problem, along with a brief discussion of the (in)famous $P \stackrel{?}{=} NP$ problem. As a bonus, we will see examples of *approximation* algorithms: if an optimization problem is too hard (e.g. NP-hard) to solve exactly, sometimes we can aim lower and give a fast algorithm that instead computes only an approximately-optimal solution.

6.1 Reductions

We start with a basic question. Suppose we have an algorithm $\mathcal{A}_P$ that solves Problem P . Can we use $\mathcal{A}_P$ to construct an algorithm $\mathcal{A}_Q$ for a different Problem Q ?

The notion of a *reduction* captures this. Figure 6.1 is a good visualization: to build an algorithm $\mathcal{A}_Q$ for Problem Q , a reduction makes blackbox calls to algorithm $\mathcal{A}_P$, with suitable additional algorithmic processing. $\mathcal{A}_Q$ can construct whatever input it wants to call with $\mathcal{A}_P$ (as long as the input can be constructed efficiently), and use the outputs also however it wants.

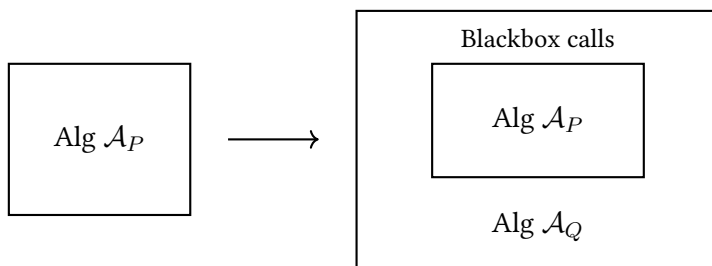


Figure 6.1. A reduction is a *wrapper*, analogous to building new software via making API calls.

In the above, we say that we have reduced Problem Q to Problem P . Reduction is a key algorithmic concept, because it allows us to leverage algorithms we already know about to solve new problems!

We next show two simple examples of reductions. In fact, we already discussed one in Chapter 2 before, without using the term. The first example is using binary search

to reduce a monotone optimization problem to a monotone decision problem, from Section 2.4. The second example we discuss is Problem 3.2 on Homework 3, which we reduce to the shortest path problem at the expense of an additional $O(\log n)$ factor in the runtime.

6.1.1 Example: Reduction from Optimization to Decision

Consider again the following pair of problems.

- **Problem Q :** Find the minimum k such that k is “feasible” according to some rule. Here, we require feasibility to be monotone in the sense that if k is feasible, then every $k' \geq k$ is also feasible.
- **Problem P :** On input k , determine whether k is “feasible”.

Problem Q is an optimization problem and Problem P is the corresponding decision problem, and we want to reduce Q to P . That is, we want to write an algorithm $\mathcal{A}_Q$ for Q , given an algorithm $\mathcal{A}_P$ for P . The reduction, as we have already seen in Chapter 2, is just to run a binary search over the possible values of k .

Algorithm 6.1. The $\mathcal{A}_Q$ Algorithm, reducing from optimization to decision

Input: An upper bound $k_{\max}$ on the answer, and blackbox access to $\mathcal{A}_P$

1. $L \leftarrow 0$; $R \leftarrow k_{\max}$
 2. while $L < R$:
 - (a) $m \leftarrow \lfloor \frac{L+R}{2} \rfloor$
 - (b) Call $\mathcal{A}_P$ to check if m is feasible
 - (c) If yes, $R \leftarrow m$
 - (d) If no, $L \leftarrow m + 1$
 3. return L
-

6.1.2 Example: The Lineland Road Trip (Homework 3, Problem 3.2)

In this next example, recall Problem 3.2 from Homework 3. The problem was intended as a dynamic programming problem, with a linear-time algorithm, but here we show how to reduce it to shortest paths, and hence solvable by Dijkstra’s algorithm at the expense of another $O(\log n)$ factor in the runtime.

SETTING: We are taking a road trip across Lineland, in one dimension. We start at position $0 = x_0$ and must end at position $x_{n+1} = 5000$. Along the way there are n motels, where motel i is at position x_i and costs p_i to stay at. We can only drive 200 miles before we have to rest, so every leg of the trip between two motels must have distance at most 200 miles. The goal is to compute the minimum total cost of getting from x_0 to x_{n+1} , where by convention we set $p_{n+1} = 0$.

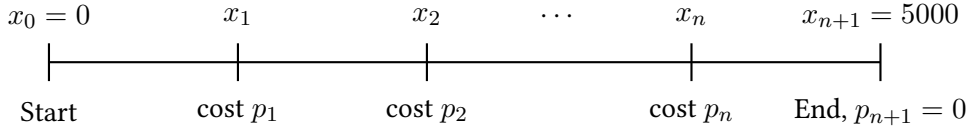


Figure 6.2. The Lineland road trip. Motels sit at positions $x_1, \dots, x_n$ with costs $p_1, \dots, p_n$, and each leg of the trip must be at most 200 miles.

The reduction is to build a graph in which paths correspond to itineraries, and then ask for a shortest path.

Algorithm 6.2. Lineland Road Trip via Shortest Paths

1. Construct the directed weighted graph $G = (V, E)$ where
 - (a) $V = \{x_0, x_1, \dots, x_n, x_{n+1}\}$
 - (b) $E = \{x_i \rightarrow x_j \text{ with weight } p_j \mid i < j \text{ and } x_j - x_i \leq 200\}$
 2. Run Dijkstra's algorithm to get the minimum distance from x_0 to x_{n+1} on G
-

The weight of the edge $x_i \rightarrow x_j$ is p_j , since we pay the cost of staying at the motel at x_j once we make the drive according to that edge.

RUNTIME: Assuming integer coordinates, the problem could have been solved with dynamic programming in $O(n)$ time, since there is only a constant number (200) of possible one-hop destinations and one-hop predecessors for each motel. On the other hand, this reduction to Dijkstra's instead yields an $O(n \log n)$ time algorithm, since the vertex set and edge set both have size $O(n)$.

CORRECTNESS: We want to show that

$$\text{minimum itinerary cost} = \text{dist}_G(x_0, x_{n+1}).$$

In order to show equality between two quantities, it is usual for us to instead prove the two inequalities separately.

Proof. $\geq$: Consider any minimum cost road trip itinerary

$$0 = x_0 \rightarrow x_{i_1} \rightarrow x_{i_2} \rightarrow \dots \rightarrow x_{n+1} = 5000.$$

The cost of this itinerary is $p_{i_1} + p_{i_2} + \dots + p_{i_k}$, where k is the number of motels stayed at. There is a corresponding path from x_0 to x_{n+1} in G with exactly the same cost, namely

$$x_0 \xrightarrow{p_{i_1}} x_{i_1} \xrightarrow{p_{i_2}} x_{i_2} \xrightarrow{p_{i_3}} \dots \xrightarrow{0} x_{n+1}.$$

The cost of this path upper bounds $\text{dist}_G(x_0, x_{n+1})$.

So

$$\begin{aligned} \text{minimum itinerary cost} &= \text{length of this path} \\ &\geq \text{dist}_G(x_0, x_{n+1}) \end{aligned}$$

$\leq$: Consider any shortest path on G ,

$$x_0 \xrightarrow{p_{i_1}} x_{i_1} \xrightarrow{p_{i_2}} \cdots \xrightarrow{0} x_{n+1}.$$

The argument is essentially the same, that there is a corresponding itinerary $x_0 \rightarrow x_{i_1} \rightarrow \cdots \rightarrow x_{n+1}$, with cost equal to the graph distance. This itinerary in turn has cost upper bounding the minimum possible cost. ■

6.2 Computational Hardness

Beyond building new algorithms, the idea of reduction also enables us to talk about the *hardness* of computational problems. Let us start with an informal question.

INFORMAL QUESTION: Suppose we can use an algorithm for Problem P to solve Problem Q , and suppose we know/conjecture that Problem Q is hard. What does that imply about Problem P ?

INFORMAL ANSWER: Problem P should also be hard. Why? Consider the contrapositive: If we cannot solve Q , then we cannot solve P . So P is a *harder* problem than Q .

We need a notion of “hardness” to make this precise. For non-mathematical purposes, here is a definition of the frequently-used term of NP-hardness.

NOTION (NP-HARD): If $P \neq \text{NP}$ (see Section 6.3 for a formal definition), then there is no polynomial time algorithm for a problem that is NP-hard.

6.2.1 How to Show NP-Hardness

Suppose we want to show that a Problem P is NP-hard. What we do is to find another problem Q that is *known* to be NP-hard, and construct an algorithm $\mathcal{A}_Q$ that

1. calls $\mathcal{A}_P$ as a blackbox polynomially many times, and
2. does only polynomial extra work.

It is easy to get confused about the direction of the reduction needed to show hardness, even for professionals. I personally memorize this: we always use a *harder* problem’s algorithm to solve an *easier* problem.

Note that ECS 120 will (likely) talk about this slightly differently, using a simpler and more restrictive notion of reductions, resulting in a slightly different notion of NP-hardness.

6.2.2 Known NP-Hard Problems

The above reduction approach requires knowing a list of NP-hard problems to begin with, so we give some basic ones here. There are really many known NP-hard problems at this point, but here are two graph problems, and we also introduce the *original* NP-hardness result.

Definition 6.1 (Independent Set) Given an undirected, unweighted graph G , find the maximum number of vertices such that no pair among them is connected by an edge.

Definition 6.2 (Clique) Given an undirected, unweighted graph G , find the maximum number of vertices such that all of them are connected to each other.

Even within graph problems, there are many other known NP-hardness results.

Next up is the original NP-hard problem of Boolean Satisfiability (SAT).

Definition 6.3 (SAT) Given a Boolean formula with n Boolean variables $x_1, \dots, x_n$, determine if there exists an assignment $x_i \mapsto \text{T/F}$ such that the formula evaluates to T.

The theorem underlying the study of NP-hardness is the Cook-Levin Theorem, which we state but will not prove.

Theorem 6.4 (Cook–Levin) SAT is NP-hard.

6.2.3 Hardness of Clique, Assuming Hardness of Independent Set

Let us see one (elementary) reduction in full for showing hardness. For this example, we will assume the NP-hardness of Independent Set, and use it to prove the NP-hardness of Clique.

Recall that we use a harder problem’s algorithm to solve an easier problem. So, to prove that Clique is hard, we have to write down an algorithm for Independent Set using blackbox calls to an algorithm for Clique.

The key observation is straightforward: a set of vertices is an independent set in G if and only if it is a clique in the complement graph constructed by flipping edges to non-edges and vice versa.

Algorithm 6.3. Solving Independent Set Using an Algorithm for Clique

1. Given $G = (V, E)$, construct the complement graph $\overline{G} = (V, \binom{V}{2} \setminus E)$
 2. Return $\text{MAXCLIQUE SIZE}(\overline{G})$
-

Clearly the reduction is computationally efficient, so it remains to prove correctness of this algorithm.

CORRECTNESS: We need to show that

$$\text{Max Independent Set Size of } G = \text{Max Clique Size of } \overline{G}.$$

Proof. $\leq$: Consider a maximum size independent set S of G . For all $u, v \in S$, there is no edge (u, v) in G , so by construction $(u, v) \in \overline{G}$. This means S is a clique in $\overline{G}$, and hence the maximum independent set size of G equals the size of *some* clique in $\overline{G}$, which is at most the maximum clique size of $\overline{G}$.

$\geq$: Consider a maximum size clique S of $\overline{G}$. By the symmetric argument to the above, S is an independent set in G . So the maximum clique size in $\overline{G}$ equals the size of *some* independent set in G , which is upper bounded by the maximum independent set size in G . ■

The above example is elementary, and hardness proofs can involve much more interesting reductions and analyses. See ECS 120 or other graduate-level courses for details.

6.3 Complexity Classes

Up till now, in this chapter, we have been using the terms P, NP without any formal definition, and defined NP-hardness based on these undefined terms. We now remedy this by stating the definitions precisely.

A *decision problem* is a Boolean function f that takes an input x and returns $f(x) \mapsto \text{T/F}$.

Definition 6.5 (Class P) P (which stands for Polynomial) is the set of decision problems computable in $\text{poly}(|x|)$ time.

There are, of course, plenty of examples of problems in P.

- Does there exist a minimum spanning tree of weight ≤ 5 ?
- Given the sticks and the targets m and k from [Section 2.4](#), can we produce at least m sticks of length exactly k ?

The class NP stands for *nondeterministic polynomial*, **not** not-deterministic polynomial. There are a few ways to define the class, but here is perhaps the way that is easiest to understand: NP is defined as the class of problems where, when given a candidate solution, we can check it efficiently.

Definition 6.6 (Class NP) A decision problem f is in NP if there exists a deterministic algorithm $V(x, w)$ and polynomials p, q such that

1. $V(x, w)$ runs in $p(|x| + |w|)$ time,
2. For all x such that $f(x) = 1$, there exists w of length $q(|x|)$ such that $V(x, w) = 1$, and
3. For all x such that $f(x) = 0$, it holds that $V(x, w) = 0$ for any w .

We call x an *instance* of the problem and w a *witness* for x . This is perhaps a bit abstract as a definition, so let us see a few examples.

SAT: An instance x is a Boolean formula, and a witness w is an assignment to the variables. We can construct a verifier $V(x, w)$ that simply checks whether the assignment makes the formula evaluate to true. A satisfiable formula x will have a satisfying assignment w , and an unsatisfiable formula will not.

3-COLOURING: Given an undirected, unweighted graph G , can we colour the vertices by $\{0, 1, 2\}$ such that for all $(u, v) \in E$, the endpoints u and v have different colours? Here an instance x is a graph $G = (V, E)$ and a witness w is a colouring $c : V \rightarrow \{0, 1, 2\}$. A verifier $V(x, w)$ checks whether the colouring violates any edge.

EVERY PROBLEM IN P: Construct $V(x, w)$ by ignoring w and just compute $f(x)$ directly. This trivially implies $P \subseteq NP$.

Again, informally, NP is the set of problems where it is possible to *check* solutions efficiently, whereas P is the set of problems where it is possible to *find* solutions efficiently. The $P \stackrel{?}{=} NP$ problem is therefore asking whether finding solutions is as easy as checking them.

We can now define NP-hardness in a more concrete way than the roundabout notion in Section 6.2, although still a bit informally to keep the exposition simple.

Definition 6.7 (NP-hard, slightly informal) A problem f is NP-hard if for every problem $g \in NP$, g can be reduced to f . That is, f is harder than every $g \in NP$.

One important aspect to emphasize about this definition is that it includes non-decision problems as well, for example optimization problems generally. So we can also talk about NP-hardness of problems beyond yes/no problems.

This is a very stringent definition, since it requires a problem to be harder than *every* problem in the class NP. Do NP-hard problems actually exist? That is in fact what the Cook-Levin theorem from Section 6.2 showed: the SAT problem is harder than every problem in NP. ECS 120 (probably?) proves this, but the key idea requires using an equivalent but very different definition of the class NP, using the notion of non-deterministic algorithms. Framing the class NP in this different way, fixing a problem $f \in NP$ and an input x , the computation of a corresponding non-deterministic algorithm can be encoded into a Boolean formula, in a way such that the formula is satisfiable if and only if $f(x) = 1$.

Finally, we can define the class NP-complete, where we restrict our attention back to decision problems again.

Definition 6.8 (NP-complete) A *decision* problem f is NP-complete if $f \in NP \cap NP\text{-hard}$.

NP-complete problems are therefore the hardest problems in NP, yielding this corollary and re-expression of the $P \stackrel{?}{=} NP$ question.

Corollary 6.9 $P = NP$ if and only if there exists an NP-complete problem with a polynomial time algorithm.

6.4 Approximation Algorithms

Unfortunately, many natural and practically useful optimization problems are NP-hard. For example:

- Maximum Independent Set
- Maximum Clique
- 0-1 Knapsack

The inclusion of 0-1 knapsack on this list should be surprising, since we derived an $O(Cn)$ time dynamic programming algorithm back in Chapter 4. However, notice that the runtime depends on the capacity C , which takes only $O(\log(C))$ bits to specify. Thus our dynamic programming algorithm is technically running in time exponential in its input size. This type of algorithm is sometimes called a *pseudo-polynomial* algorithm, where the runtime is polynomial in the *value* of the input not its representation size.

So what should we do if we are faced with an NP-hard optimization problem we have to solve?

LIFE LESSON: If you can't beat the game, *change* the game.

One (of many) way out is to lower our standards a little. Instead of demanding an optimal solution, what if we only try computing a solution that is not too far from optimal? This leads to the study of *approximation* algorithms.

6.4.1 A 2-Approximation for 0-1 Knapsack

Let us see an example, for the 0-1 knapsack problem. We will aim to achieve a 2-approximation, namely finding an item subset whose reward value is no less than half of the optimal possible reward.

Throughout this section, we also assume without loss of generality that every item satisfies $w_i \leq C$. Any algorithm can be preprocessed by filtering out items that are heavier than C .

The following is a rather hacky greedy algorithm, but which turns out to yield 2-approximation.

Algorithm 6.4. 2-Approximation for 0-1 Knapsack

1. Sort items in non-increasing order of r_i/w_i
 2. Compute the smallest i such that $\sum_{j=1}^i w_j > C$
 3. Pick the better of $\sum_{j=1}^{i-1} r_j$ and r_i
-

In other words, take items in decreasing order of reward density, like in fractional knapsack, until the next one would not fit. While it is tempting to return this solution, the strange twist in the algorithm is to compare this greedy solution with just taking

the next item by itself¹. Both options have weight $\leq C$ by construction or assumption, and so it is legal to return the maximum of either option. Note, as a corollary, that we could replace r_i in Step 3 by $r_{\max}$, the largest reward of any single item.

Theorem 6.10 *If OPT is the optimal reward, then Algorithm 6.4 achieves objective value at least OPT/2.*

Proof. The idea is to compare Algorithm 6.4's solution with the *fractional* knapsack optimum (see Chapter 5 again for the fractional knapsack example).

Let OPT_{frac} be the optimum for fractional knapsack on the same instance, achieved (after sorting items by reward density) by

$$\rho_1 = \dots = \rho_{i-1} = 1, \quad \rho_i = \frac{C - \sum_{j=1}^{i-1} w_j}{w_i},$$

with $\rho_j = 0$ for $j > i$. So

$$\sum_{j=1}^{i-1} r_j + \rho_i r_i = \text{OPT}_{\text{frac}} \geq \text{OPT},$$

where the inequality holds because any 0-1 solution is in particular a fractional solution.

Now, by an averaging argument, it must be the case that

$$\max \left(\sum_{j=1}^{i-1} r_j, \rho_i r_i \right) \geq \frac{1}{2} \text{OPT}.$$

Finally, since $\rho_i \leq 1$, we have

$$\max \left(\sum_{j=1}^{i-1} r_j, r_i \right) \geq \frac{1}{2} \text{OPT}.$$

■

6.4.2 Fully Polynomial Time Approximation Schemes

It turns out we can do even better than a 2-approximation for the 0-1 knapsack problem.

Definition 6.11 (Fully Polynomial Time Approximation Scheme) A fully polynomial time approximation scheme (FPTAS) is such that, given any $\epsilon > 0$ and an input of size n , finds a $(1 - \epsilon)$ -approximate solution in $\text{poly}(\frac{1}{\epsilon}, n)$ time.

For 0-1 knapsack, where again we assume every item has weight at most the capacity C , the idea is to use a *different* dynamic programming algorithm from the one in Chapter 4, one that instead assumes integer rewards and real-valued weights, plus the idea of rounding. Assuming n items and a maximum integer reward of $r_{\max}$, the desired dynamic programming algorithm runs in $O(r_{\max} \text{poly}(n))$ time.

¹It is possible to construct examples where taking only the greedy solution does not yield a 2-approximation.

TABLE ENTRIES: $T[i, r]$ is the minimum weight of a subset of items $1, \dots, i$ with reward at least r .

RECURRENCE:

$$T[i, r] = \min(T[i-1, r], w_i + T[i-1, r - r_i])$$

TABLE SIZE: The table has n rows and one column per achievable reward value, so its size is $n \times \sum_i r_i \leq n \times n \times r_{\max}$.

ROUNDING: How do we use the above integer-reward dynamic programming algorithm? The immediate idea is to round our given instance's rewards to a multiple of some granularity parameter k .

One key constraint, due to the desired runtime, is to make sure that the number of possible rounded integer rewards is upper bounded by $\text{poly}(\frac{1}{\epsilon}, n)$. Concretely, if we now reuse the notation $r_{\max}$ for the maximum real-valued reward in the input instance, we choose

$$k = \frac{\epsilon r_{\max}}{n},$$

and then consider the rounded rewards

$$r'_i = \left\lceil \frac{r_i}{k} \right\rceil k.$$

Every rounded reward is now a multiple of k , so the number of distinct reward values is at most $r_{\max}/k = \frac{n}{\epsilon}$.

RUNTIME: The table has size upper bounded by $n \cdot n \cdot O(n/\epsilon) = O(n^3/\epsilon)$, with constant work per table entry, so the overall runtime is $O(n^3/\epsilon)$. This is clearly a fully polynomial runtime as desired.

APPROXIMATION RATIO: The approximation ratio can be calculated to be $1 - \epsilon$, as a simple exercise: (1) Consider an optimal solution for the unrounded problem, argue about its total reward using the rounded item rewards, (2) observing that the dynamic programming algorithm returns a solution that is no worse in the rounded rewards and (3) lower bound this solution's unrounded total reward in terms of its rounded total reward.

6.5 Inapproximability

Unfortunately, efficient approximation is not always possible for hard optimization problems. Some NP-hard optimization problems have no good+fast approximation algorithm at all, and we can prove that (assuming $P \neq NP$).

6.5.1 The Travelling Salesman Problem

Definition 6.12 (Hamiltonian cycle) A Hamiltonian cycle on a graph is one that visits every vertex *exactly* once.

Definition 6.13 (Travelling Salesman Problem (TSP)) Given an undirected weighted graph G , find a minimum weight “Hamiltonian” cycle.

It is NP-hard in general to find or determine the existence of a Hamiltonian cycle, so TSP is also NP-hard. But it turns out TSP is so hard that there is not even a half-decent approximation algorithm.

6.5.2 Inapproximability of TSP

Theorem 6.14 (Inapproximability of TSP) *For any $\alpha \in (1, 2^{\text{poly}(n)})$, there is no polynomial time α -approximation algorithm for TSP unless $P = NP$.²*

Proof. Suppose for the sake of contradiction that there is a polynomial time α -approximation algorithm for TSP. We will use it to decide whether an arbitrary graph has a Hamiltonian cycle, which as discussed is itself an NP-hard problem.

Given an unweighted graph G , construct a new graph $G' = (V', E')$ where $V' = V$ and E' contains

- E , with weight 1 on every edge, and
- $\binom{V}{2} \setminus E$, with weight $\alpha n + 2$ on every edge.

So G' is a complete graph, where the original edges of G are light and all the non-edges of G are extremely heavy.

If G has a Hamiltonian cycle, then we claim that the TSP tour of G' must be a Hamiltonian cycle on G . This is because any single non- G edge adds $\alpha n + 1$ to the cost of the tour, which already exceeds α times the length of a Hamiltonian cycle using only G edges, which is αn . So an α -approximate TSP tour cannot use even a single non- G edge.

On the other hand, if G has no Hamiltonian cycle, then every Hamiltonian cycle in the complete graph G' uses at least one edge that was not in G .

So we have derived a new polynomial time algorithm for checking the existence of Hamiltonian cycles, assuming an α -approximate TSP algorithm, contradicting the assumption that $P \neq NP$. ■

6.5.3 Metric TSP and a 2-Approximation

But TSP is indeed a real-world problem that requires solving. What else can we do?

ANOTHER LIFE/PROBLEM SOLVING LESSON: If you can't beat the game, look for rules or assumptions you missed.

An important assumption we missed is that, in real-world logistics, edge weights cannot be arbitrary positive numbers. If the vertices are points in physical space, their distances must form a *metric* (a formal mathematical term) and satisfy the triangle inequality. So let us now restrict attention to *metric TSP*.

²Morally, the result holds for any $\alpha > 1$, but once α is too large, the reduction runs into issues such as the representation size of the constructed instances, requiring too many bits even to specify. That is why we restrict attention to $\alpha \leq \exp(\text{poly}(n))$.

Definition 6.15 (Metric TSP) A metric TSP instance is a complete graph with edge weights satisfying the triangle inequality:

$$w_{ik} \leq w_{ij} + w_{jk} \quad \text{for all vertices } i, j, k.$$

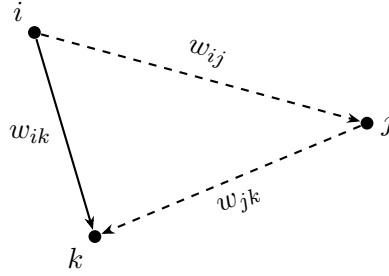


Figure 6.3. The triangle inequality. The direct route from i to k , drawn solid, is never more expensive than the dashed detour via j .

With the metric assumption, we can now give a 2-approximation algorithm.

Algorithm 6.5. 2-Approximation for Metric TSP

1. Compute a minimum spanning tree T of graph G
 2. Compute a DFS tour of T (starting at any vertex as tree root)
 3. Turn the DFS tour into a Hamiltonian cycle by skipping visited vertices
-

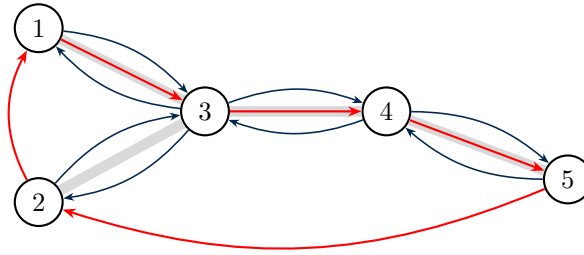


Figure 6.4. Consider the DFS tour $1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 3 \rightarrow 1$. Shortcutting already-visited vertices gives the tour $1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 2 \rightarrow 1$, by the two shortcuts $5 \rightarrow 2$ and $2 \rightarrow 1$.

Theorem 6.16 *Algorithm 6.5 returns a Hamiltonian cycle with cost at most 2 times the minimum TSP tour.*

Proof. Consider a minimum TSP tour and delete any edge from it. What remains is a path that visits every vertex, which in particular is a spanning tree. So

$$w(\text{minimum TSP tour}) \geq w(\text{minimum spanning tree}).$$

On the other hand, the algorithm's output tour has weight at most that of the DFS tour, since Step 3 shortcuts any visited vertices, and the triangle inequality (plus induction) guarantees that the shortcutting cannot increase the tour weight. Finally, the DFS tour has weight twice the weight of the computed minimum spanning tree, since every edge is used exactly twice in a DFS tour of any tree. So the algorithm's output tour has weight at most $2 \cdot w(\text{minimum spanning tree})$, showing the 2-approximation. ■